山东大学 郑艳伟

编译原理与技术——题库

作者：郑艳伟

组织：山东大学

时间：April 6, 2024–March 13, 2026

版本：1.0

山东大学

依托教育部编译原理课程虚拟教研室第二批“编译课程教学试点项目”制作

说明

“教育部编译课程虚拟教研室”教学改革项目“编译课程习题库建设”（2023-003）
配套教材：郑艳伟，于东晓，周劲. 编译原理与技术（微课版），北京：清华大学出版社，2025.

121 个算法，125 个翻译模式，从零手搓编译器。
纸质版 894 千字，加电子版 954 千字。

图 1 配套教材

山东大学郑艳伟

郑艳伟
Jul 4, 2025

目录

1 编译器概述 1

第 1 章 练习 1

2 文法与语言设计 4

第 2 章 练习 4

3 词法分析 6

第 3 章 练习 6

4 语法分析 12

第 4 章 练习 12

5 符号表 21

第 5 章 练习 21

6 运行时存储空间组织 23

第 6 章 练习 23

7 语法制导翻译与中间代码生成 27

第 7 章 练习 27

8 中间代码优化 32

第 8 章 练习 32

9 目标代码生成 39

第 9 章 练习 39

10 面向对象语言的翻译 44

第 10 章 练习 44

山东大学 郑艳伟

第 1 章 编译器概述

第 1 章 练习

1. 简述自然语言和形式语言的区别。
2. 根据与机器硬件的紧密程度，程序语言可以分为哪些类别？
3. 试说明什么是翻译程序、编译程序、解释程序和汇编程序。
4. 列举你所知道的编译型语言和解释型语言。
5. 试说明什么是运行时和编译时。
6. 试说明什么是宿主机和目标机。李华在一台安装了 Windows 操作系统的 A 机器上，开发一个在鸿蒙手机 B 上运行的程序，机器 A 和 B 哪个是宿主机，哪个是目标机？
7. 李华设计了一种新语言，起名为 Y 语言，想用 x86 机器、Windows 操作系统上的 C 语言编译器实现它，试说明其实现过程，并画 T 型图表示。
8. 已有 x86 机器、Windows 操作系统上的 C 语言编译器，李华试图将其移植到鸿蒙操作系统上，试说明其实现过程，并画 T 型图表示。
9. 李华设计了一种新语言，起名为 Y 语言，想在 x86 机器、Windows 操作系统上，采用自举（自编译）方式实现，试说明其实现过程。
10. 从语言范式角度，说明高级程序设计语言的分类。
11. 列举你所知道的命令式语言和声明式语言。
12. 列举常见的高级程序设计语言结构。
13. 面向过程的语言和面向对象的语言，它们的主要特征是什么？
14. 基于对象的语言和面向对象的语言，它们的主要区别是什么？
15. 动态语言的主要特征是什么？
16. 过程嵌套定义的语言，其主要特征是什么？
17. 调研程序闭包的概念，并加以说明。列举几种支持程序闭包的语言。
18. 什么是字节对齐？为什么要进行字节对齐？
19. 假设从地址 1000h 处开始分配空间，试写出如下 C 风格声明语句中，各变量分配的地址：int i1; bool b1; bool b2; int i2; short s1; int i3;。
20. 列举至少 6 种常见高级语言语句形式。
21. 目标代码有哪些形式？
22. 列举常见的 CPU 架构，并说明它们采用哪种指令集，列举使用这些架构的厂商。
23. 画图并解释运行时存储空间的区域划分。
24. 写出 x86 的所有通用寄存器。
25. 在进行地址计算时，lea 指令相对于 mov 指令有哪些优点？
26. 写出如下指令：
    (1) 将地址 0ABCDh 取到寄存器 EAX；
    (2) 将静态数据区变量 x 的地址取到寄存器 EAX；
27. 编写汇编程序：
    (1) 在静态数据区声明 3 个 32 位变量 a、b、c，其中 a、b 初始化为 10 和 20，c 不初始化；

山东大学 郑艳伟

(2) 在代码区编写代码，将 a+b 的结果保存到变量 c。

28. 编写汇编程序：
    (1) 在静态数据区声明 3 个 float 类型变量 a、b、c，其中 a、b 初始化为 3.14 和 2.0，c 不初始化；
    (2) 在代码区编写代码，将 a*b 的结果保存到变量 c。
29. 编写汇编程序：
    (1) 在静态数据区声明 2 个 float 类型变量 a、b，1 个 32 位整型变量 c，其中 a、b 初始化为 3.14 和 2.0，c 不初始化；
    (2) 在代码区编写代码，将 a*b 的结果取整后保存到变量 c。
30. 计算 a/b，其中 a 和 b 都是静态数据区的无符号整型变量，将商和余数分别保存到 ESI 和 EDI 寄存器，试写出汇编代码。
31. 计算 a/b，其中 a 和 b 都是静态数据区的有符号整型变量，将商和余数分别保存到 ESI 和 EDI 寄存器，试写出汇编代码。
32. 编写一个汇编过程，实现从 1 加和到一个数字 n，结果保存到 result，其中 n 和 result 已经在静态数据区声明和初始化，要求保护使用的寄存器。
33. 编写一个汇编过程，实现一个数字 n 的阶乘，结果保存到 result，其中 n 和 result 已经在静态数据区声明和初始化，要求保护使用的寄存器。
34. 在静态数据区，声明了两个 real8 变量 x 和 y 并已初始化，另外声明了一个字节变量 result，比较 x>y，如果为真将 result 置为 1，否则置为 0。
35. 在静态数据区，声明了 3 个 real8 变量 a、b 和 c，分别是方程 $ax^2+bx+c=0$ 的系数，已初始化；另外声明了 1 个字节变量 hasRoot，以及 2 个 real8 变量 x1 和 x2，未初始化。如果方程有解，则将 hasRoot 置为 1，且将两个根写入到 x1 和 x2；如果方程无解，则将 hasRoot 置为 0。
36. 乘法指令比较耗时，一般一条乘法指令的执行时间是加法的几倍到十几倍。有乘法指令“imul eax, 70”，使用移位指令 shl 及 mov、加减法指令等，实现与该乘法指令相同的结果。
37. 在静态数据区，声明了 3 个 real8 变量 a、b 和 c，分别是函数 $f(x)=ax^2+bx+c\ (a>0)$ 的系数，已初始化；另外声明了 1 个 real8 变量 yMin，未初始化。编写汇编程序，使用牛顿法求函数 $f(x)$ 的最小值，并将结果写入 yMin。
38. 在静态数据区，声明了 3 个 real8 变量 a、b 和 c，分别是函数 $f(x)=ax^2+bx+c\ (a>0)$ 的系数，已初始化；另外声明了 1 个 real8 变量 yMin，未初始化。编写汇编程序，使用梯度下降法求函数 $f(x)$ 的最小值，并将结果写入 yMin。
39. 在静态数据区，声明了 3 个 real8 变量 a、b 和 c，分别是函数 $f(x)=ax^2+bx+c\ (a\neq 0)$ 的系数，已初始化；另外声明了 2 个 real8 变量 xExtremum 和 yExtremum，未初始化。编写汇编程序，使用牛顿法求函数 $f(x)$ 的极值，并将取得极值时的 x 和 y 分别写入 xExtremum 和 yExtremum（提示：如果 a>0，求 $f(x)$ 的最小值；如果 a<0，，求 $f(x)$ 的最大值，相当于求 $-f(x)$ 的最小值）。
40. 在静态数据区，声明了 3 个 real8 变量 a、b 和 c，分别是函数 $f(x)=ax^2+bx+c\ (a\neq 0)$ 的系数，已初始化；另外声明了 2 个 real8 变量 xExtremum 和 yExtremum，未初始化。编写汇编程序，使用递归下降法求函数 $f(x)$ 的极值，并将取得极值时的 x 和 y 分别写入 xExtremum 和 yExtremum（提示：如果 a>0，求 $f(x)$ 的最小值；如果 a<0，，求 $f(x)$ 的最大值，相当于

山东大学 郑艳伟

求 $-f(x)$ 的最小值）。

41. 画图表示中间语言的分类。
42. 求表达式 “$(a+b)*(a-b)$” 的后缀式。
43. 求表达式 “$(a+b)\hat{}(a-b)\hat{}c$” 的后缀式。
44. 求表达式 “$a \vee b \neq c+d \wedge a*b < f$” 的后缀式。
45. 写出赋值语句 “$x = a+b+(a+b)*(a-b)$” 的四元式序列。
46. 写出赋值语句 “$x = a+b+(a+b)*(a-b)$” 的三元式序列。
47. 写出赋值语句 “$x = a+b+(a+b)*(a-b)$” 的间接三元式序列。
48. 写出赋值语句 “$x = (a+b)\hat{}(a-b)\hat{}(a+b)$” 的四元式序列。
49. 写出赋值语句 “$x = (a+b)\hat{}(a-b)\hat{}(a+b)$” 的三元式序列。
50. 写出赋值语句 “$x = (a+b)\hat{}(a-b)\hat{}(a+b)$” 的间接三元式序列。
51. 写出赋值语句 “$x = a \vee b \neq c+d \wedge a*b < c+d$” 的四元式序列。
52. 写出赋值语句 “$x = a \vee b \neq c+d \wedge a*b < c+d$” 的三元式序列。
53. 写出赋值语句 “$x = a \vee b \neq c+d \wedge a*b < c+d$” 的间接三元式序列。
54. 画图表示编译器的构成，并说明各部分功能。
55. 试说明编译器前端和后端。

山东大学 郑艳伟

第 2 章 文法与语言设计

第 2 章 练习

1. 在词法分析和语法分析阶段，符号和符号串各代表什么？
2. 设 $\Sigma = 0, 1$ 上的符号串 $x = 00, y = 11$，写出如下表达式的结果：xy、yxy、x^3、x^2y^2、$|yx|$。
3. 设 $\Sigma = 0, 1$ 上的符号串 $x = 00, y = 11$，写出 xy 的前缀和后缀。
4. 设 $\Sigma = 0, 1$ 上的符号串集合 $X = 00, 11, Y = 01, 10$，写出 XY、YX、X^2、$X\varepsilon$。
5. 设 $x = 001011$
   (1) 写出 x 的长度不超过 3 的前缀构成的集合 U；
   (2) 写出 x 的长度不超过 3 的后缀构成的集合 V；
   (3) 求 UV 和 VU；
   (4) 求 U^2 和 V^2。
6. 文法 $G[E] : E \to EAE|i, A \to +|-|*|/$。
   (1) 采用规范推导，从文法的开始符号推导出句子 $i + i * i$；
   (2) 采用规范归约，将句子 $i - i/i$ 归约到文法的开始符号；
   (3) 写出文法表示的语言；
   (4) 判断该文法是否为二义文法，并说明理由。
7. 文法 $G[E] : E \to E \wedge E|E \vee E|(E)|i$，其中 i 表示任意布尔型变量。
   (1) 采用规范推导，从文法的开始符号推导出句子 $i \wedge (i \vee i)$；
   (2) 采用规范归约，将句子 $i \vee (i \wedge i)$ 归约到文法的开始符号；
   (3) 写出文法表示的语言；
   (4) 判断该文法是否为二义文法，并说明理由。
8. 文法 $G[E] : E \to E + T|T, T \to T * F|F, F \to (E)|i$。
   (1) 写出句型 $i * (E)$ 的短语、直接短语和句柄；
   (2) 写出句子 $i * (i + i)$ 的短语、直接短语和句柄。
9. 文法 $G[E] : E \to E \vee T|T, T \to T \wedge F|F, F \to \neg F|(E)|i$。
   (1) 写出句型 $i \wedge (E)$ 的短语、直接短语和句柄；
   (2) 写出句子 $i \wedge \neg(i \vee i)$ 的短语、直接短语和句柄。
10. 文法 $G[E] : E \to (A, E, E)|i, A \to +|-|*|/$。
    (1) 类 Lisp 程序 (θ, a, b) 表示 $a\theta b$ 的结果，其中 θ 为运算符，试写出 $a * (b + c)/d$ 的类 Lisp 程序形式；
    (2) 任意变量与 i 匹配，试采用规范推导从开始符号推导出上述句子；
    (3) 采用规范归约将以上句子归约到开始符号；
    (4) 画出上述句子的语法树；
    (5) 写出上述句子的短语、直接短语和句柄。
11. 有文法 $G[S] : S \to AB, A \to Aa|b, B \to bB|a$，试确定该文法定义的语言。
12. 有文法 $G[S] : S \to (S)|i$，试确定该文法定义的语言。
13. 有文法 $G[S] : S \to (S)|\varepsilon$，试确定该文法定义的语言。

山东大学 郑艳伟

14. 构造文法，使其描述的语言为正偶数集合。
15. 上下文有关文法和上下文无关文法的产生式形式各是什么？主要区别是什么？
16. 构造正规式，使其表示“能被 4 整除的二进制数”，并写出其对应的上下文无关文法。
17. 构造正规式，使其表示“能被 4 整除的十进制数”，并写出其对应的上下文无关文法。
18. 分别写出正规式 α^*、$\alpha\beta^*$ 和 $\alpha^*\beta$ 的上下文无关文法。
19. $\Sigma = \{0, 1\}$，构造正规式，使其表示的字长度为奇数，且 0 和 1 交替出现，如：0, 1, 010, 101, 01010, 10101, ……；并设计上下文无关文法表示该正规式。
20. 设计上下文无关文法表示正规式 $(00|11|(01|10)^*)^*$。
21. 如果数组元素形式为 $id[num][num]\cdots[num]$，其中 id 为数组名字，num 为下标，试写出其正规式，并构造对应的上下文无关文法。
22. 考虑改造 C 风格函数定义的形式如“int Fun(int a, b; float x, y)”，该函数返回值为 int 型，有两个 int 型和两个 float 型形参。仅考虑 int 和 float 两种数据类型，函数可以没有返回值，试写出该函数定义的正规式，并构造其对应的上下文无关文法。
23. 假设有一种类 LISP 语言，其形式为 (θ, E, E)，其中 θ 为任意运算符，E 或者是运算量 i，或者是形如 (θ, E, E) 的表达式，试构造其文法。
24. 设计无二义文法，使其能够表示逻辑表达式，逻辑表达式中涉及的运算量都是布尔型变量，用终结符号 i 表示。
25. 设计无二义文法，使其能够表示逻辑表达式，逻辑表达式中涉及的运算量或者是布尔型变量 i，或者是形如 $i\theta i$ 的关系运算；关系运算中的 θ 是关系运算符，i 为整型或实型变量。
26. 设计无二义文法，使其能够表示逻辑表达式，逻辑表达式中涉及的运算量或者是布尔型变量 i，或者是形如 $E\theta E$ 的关系运算；关系运算中的 θ 是关系运算符，E 为算术表达式，仅考虑加、乘、括号运算。
27. 文法 $G = (\{S, A, B\}, \{i, x, (,)\}, P, S)$，其中 $P = \{S \to (S)|i|A, A \to xAx, B \to xBx|i\}$，试消除无用产生式。
28. 文法 $G = (\{S, A, B\}, \{i, x, (,)\}, P, S)$，其中 $P = \{S \to (S)|i|AB, A \to xAx, B \to xBx|i\}$，试消除无用产生式。
29. 文法 $G = (\{S, A, B\}, \{o, s, (,)\}, P, S)$，其中 $P = \{S \to (A), A \to S|B, B \to oBo|s\}$，试消除单非产生式。
30. 文法 $G = (\{S, A, B\}, \{o, s, (,)\}, P, S)$，其中 $P = \{S \to (A)|A, A \to S|B, B \to oBo|s\}$，试消除单非产生式。
31. 文法 $G = (\{S, A, B\}, \{o, s, (,)\}, P, S)$，其中 $P = \{S \to (A), A \to S|B, B \to oBs|\varepsilon\}$，试消除或规范空符产生式。
32. 文法 $G = (\{S, A, B\}, \{o, s, (,)\}, P, S)$，其中 $P = \{S \to (A)|A, A \to S|B, B \to oBs|\varepsilon\}$，试消除或规范空符产生式。

山东大学 郑艳伟

第 3 章 词法分析

第 3 章 练习

1. 词法分析的任务是什么？
2. 单词的类别有哪些？
3. 单词符号如何分类和编码？
4. 简述词法分析器的两种实现形式及优缺点。
5. 简述词法分析的过滤阶段的功能，它是否可以与扫描功能合并？
6. 说明双缓冲策略的意义和实现方法。
7. 什么是超前搜索？举例说明。
8. 设计 $\Sigma = \{0, 1, \cdots, 9\}$ 上的 DFA M，使其接受能被 3 整除的十进制数字。
9. 设计 $\Sigma = \{0, 1\}$ 上的 DFA M，使其接受能被 3 整除的二进制数字。
10. 设计 $\Sigma = \{0, 1\}$ 上的 DFA M，使其能识别偶数个 0 偶数个 1 的字。
11. 设计 $\Sigma = \{0, 1, \cdots, 9\}$ 上的 DFA M，使其接受能被 4 整除的十进制数字。
12. 设计 $\Sigma = \{0, 1\}$ 上的 DFA M，使其接受能被 4 整除的二进制数字。
13. 设计 DFA M，使其能识别 C 语言中的注释。
14. 设计 DFA M，使其能识别 C 语言中的十六进制数字。
15. 设计 DFA M，使其能识别 C 语言中所有形式的浮点数字（不含正负号）。
16. 编写一个对 C 语言源程序预处理的程序，删除其中的注释。
17. 正规式：$aa(a|b)^*bb$
    (1) 构造 NFA，并单符化；
    (2) 确定化；
    (3) 最小化。
18. 正规式：$b(aa)^*ba(bb)^*a$
    (1) 构造 NFA，并单符化；
    (2) 确定化；
    (3) 最小化。
19. 正规式：$(0|1)^*(01|10)(0|1)^*$
    (1) 构造 NFA，并单符化；
    (2) 确定化；
    (3) 最小化。
20. 正规式：$(a|b)(a|b|0|1)^*$
    (1) 构造 NFA，并单符化；
    (2) 确定化；
    (3) 最小化。
21. 正规式：$(0|1)^+.(0|1)^+$
    (1) 构造 NFA，并单符化；
    (2) 确定化；

山东大学 郑艳伟

(3) 最小化。

22. C 风格变量声明的正规式：$(i|r)v(,v)^*$；其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名。
    (1) 构造 NFA，并单符化；
    (2) 确定化；
    (3) 最小化。
23. Pascal 风格变量声明的正规式：$v(,v)^* : (i|r)$；其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名。
    (1) 构造 NFA，并单符化；
    (2) 确定化；
    (3) 最小化。
24. 数组元素引用的正规式：$v[(v,)^*v]$；其中 v 表示变量名或数组名。
    (1) 构造 NFA，并单符化；
    (2) 确定化；
    (3) 最小化。
25. C 风格过程调用的正规式：$v\backslash(v(,v)^*|\varepsilon\backslash)$；其中 v 表示变量名或过程名，\ 表示转义字符符号，即 \(和 \) 分别表示正规式中出现了左圆括号和右圆括号，这两个括号符号不是正规式中的元符号。
    (1) 构造 NFA，并单符化；
    (2) 确定化；
    (3) 最小化。
26. C 风格变量声明的文法 $G[S]$，产生式如下所示，其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，构造与其等价的 FA。

    $S \to iV$　　$S \to rV$　　$V \to vL$　　$L \to ,V$　　$L \to \varepsilon$

27. C 风格变量声明的文法 $G[S]$，产生式如下所示，其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，构造与其等价的 FA。

    $S \to Tv$　　$S \to Av$　　$A \to S,$　　$T \to i$　　$T \to r$

28. Pascal 风格变量声明的文法 $G[S]$，产生式如下所示，其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，构造与其等价的 FA。

    $S \to vA$　　$A \to ,B$　　$B \to vA$
    $A \to :T$　　$T \to i$　　$T \to r$

29. Pascal 风格变量声明的文法 $G[S]$，产生式如下所示，其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，构造与其等价的 FA。

    $S \to Ai$　　$S \to Ar$　　$A \to B:$
    $B \to v$　　$B \to Uv$　　$U \to B,$

30. 表示数组元素引用的文法 $G[S]$，产生式如下所示，其中 v 表示变量名或数组名，构造与其等价的 FA。

    $S \to vA$　　$A \to [L$　　$L \to vV$

山东大学 郑艳伟

$L \to vR$ $\quad V \to ,L$ $\quad R \to]$

31. 表示数组元素引用的文法 $G[S]$，产生式如下所示，其中 v 表示变量名或数组名，构造与其等价的 FA。

$S \to R]$ $\quad R \to Lv$ $\quad L \to R,$ $\quad L \to A[$ $\quad A \to v$

32. C 风格过程调用的文法 $G[S]$，产生式如下所示，其中 v 表示变量名或过程名，构造与其等价的 FA。

$S \to vA$ $\quad A \to (L$ $\quad A \to (R$ $\quad L \to vV$

$L \to vR$ $\quad V \to ,L$ $\quad R \to)$

33. C 风格过程调用的文法 $G[S]$，产生式如下所示，其中 v 表示变量名或过程名，构造与其等价的 FA。

$S \to R)$ $\quad S \to B)$ $\quad R \to Lv$ $\quad L \to R,$

$L \to A($ $\quad A \to v$ $\quad B \to A($

34. 将图3.1表示的 FA 转换为右线性文法 G_R。
35. 将图3.1表示的 FA 转换为左线性文法 G_L。

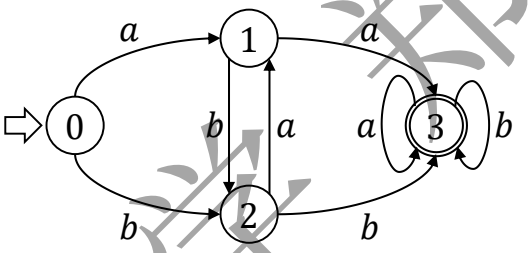


图 3.1 确定有限自动机

36. 将图3.2表示的 FA 转换为右线性文法 G_R。
37. 将图3.2表示的 FA 转换为左线性文法 G_L。

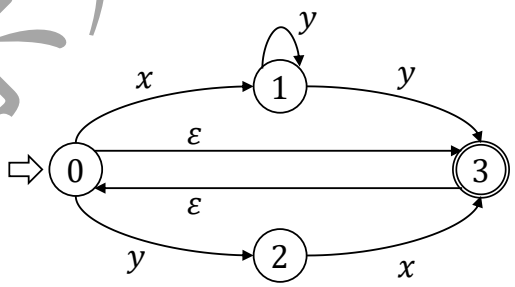


图 3.2 有限自动机

38. 将图3.3表示的 FA 转换为右线性文法 G_R。
39. 将图3.3表示的 FA 转换为左线性文法 G_L。
40. 将图3.4表示的 FA 转换为右线性文法 G_R。
41. 将图3.4表示的 FA 转换为左线性文法 G_L。
42. 将表3.9表示的 FA 转换为右线性文法 G_R。
43. 将表3.9表示的 FA 转换为左线性文法 G_L。

山东大学 郑艳伟

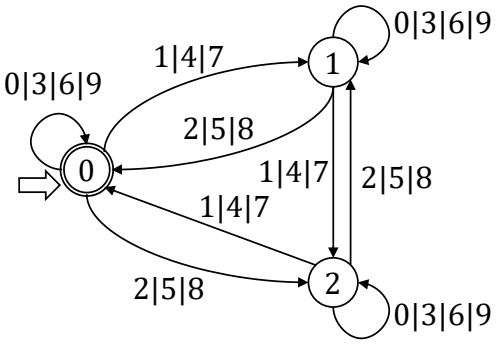


图 3.3 有限自动机

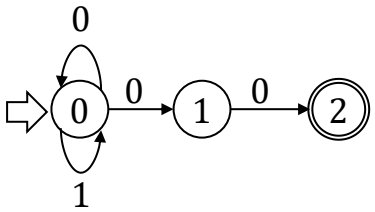


图 3.4 有限自动机

表 3.9 接受能被 4 整除的十进制数的 DFA

状态	0|4|8	1|5|9	2|6	3|7
0	0	1	2	3
1	2	3	0	1
2	0	1	2	3
3	2	3	0	1

44. 将正规式 $(00|11|(01|10)^*)^*$ 转换为右线性文法 G_R。
45. 将正规式 $(00|11|(01|10)^*)^*$ 转换为左线性文法 G_L。
46. 将正规式 $b(aa)^*(bb)^*a$ 转换为右线性文法 G_R。
47. 将正规式 $b(aa)^*(bb)^*a$ 转换为左线性文法 G_L。
48. C 风格变量声明的正规式：$(i|r)v(,v)^*$；其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，将其转换为右线性文法 G_R。
49. C 风格变量声明的正规式：$(i|r)v(,v)^*$；其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，将其转换为左线性文法 G_L。
50. Pascal 风格变量声明的正规式：$v(,v)^*:(i|r)$；其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，将其转换为右线性文法 G_R。
51. Pascal 风格变量声明的正规式：$v(,v)^*:(i|r)$；其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，将其转换为左线性文法 G_L。
52. 数组元素引用的正规式：$v[v(,v)^*]$；其中 v 表示变量名或数组名，将其转换为右线性文法 G_R。
53. 数组元素引用的正规式：$v[v(,v)^*]$；其中 v 表示变量名或数组名，将其转换为左线性文法 G_L。
54. C 风格过程调用的正规式：$v\backslash(v(,v)^*|\varepsilon\backslash)$；其中 v 表示变量名或过程名，\ 表示转义字符符号，即 \(和 \) 分别表示正规式中出现了左圆括号和右圆括号，这两个括号符号不是正规式中的元符号，将其转换为右线性文法 G_R。

山东大学 郑艳伟

55. C 风格过程调用的正规式：$v\backslash(v(,v)^*|\varepsilon\backslash)$；其中 v 表示变量名或过程名，\ 表示转义字符符号，即 \(和 \) 分别表示正规式中出现了左圆括号和右圆括号，这两个括号符号不是正规式中的元符号，将其转换为左线性文法 G_L。
56. C 风格变量声明的文法 $G[S]$，产生式如下所示，其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，构造与其等价的正规式。

    $S \to iV$　　$S \to rV$　　$V \to vL$　　$L \to ,V$　　$L \to \varepsilon$

57. C 风格变量声明的文法 $G[S]$，产生式如下所示，其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，构造与其等价的正规式。

    $S \to Tv$　　$S \to Av$　　$A \to S,$　　$T \to i$　　$T \to r$

58. Pascal 风格变量声明的文法 $G[S]$，产生式如下所示，其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，构造与其等价的正规式。

    $S \to vA$　　$A \to ,B$　　$B \to vA$
    $A \to :T$　　$T \to i$　　$T \to r$

59. Pascal 风格变量声明的文法 $G[S]$，产生式如下所示，其中 i 和 r 分别表示整型和实型数据类型，v 表示变量名，构造与其等价的正规式。

    $S \to Ai$　　$S \to Ar$　　$A \to B:$
    $B \to v$　　$B \to Uv$　　$U \to B,$

60. 表示数组元素引用的文法 $G[S]$，产生式如下所示，其中 v 表示变量名或数组名，构造与其等价的正规式。

    $S \to vA$　　$A \to [L$　　$L \to vV$
    $L \to vR$　　$V \to ,L$　　$R \to]$

61. 表示数组元素引用的文法 $G[S]$，产生式如下所示，其中 v 表示变量名或数组名，构造与其等价的正规式。

    $S \to R]$　　$R \to Lv$　　$L \to R,$　　$L \to A[$　　$A \to v$

62. C 风格过程调用的文法 $G[S]$，产生式如下所示，其中 v 表示变量名或过程名，构造与其等价的正规式。

    $S \to vA$　　$A \to (L$　　$A \to (R$　　$L \to vV$
    $L \to vR$　　$V \to ,L$　　$R \to)$

63. C 风格过程调用的文法 $G[S]$，产生式如下所示，其中 v 表示变量名或过程名，构造与其等价的正规式。

    $S \to R)$　　$S \to B)$　　$R \to Lv$　　$L \to R,$
    $L \to A($　　$A \to v$　　$B \to A($

64. 将图3.1表示的 FA 转换为正规式。
65. 将图3.2表示的 FA 转换为正规式。
66. 将图3.3表示的 FA 转换为正规式。
67. 将图3.4表示的 FA 转换为正规式。
68. 将表3.9表示的 FA 转换为正规式。

山东大学 郑艳伟

69. 写出代码3.1经过词法分析后得到的单词二元组序列。

代码 3.1 计算阶乘

```
/* 计算阶乘 **********
** 输入n，返回n! ****/
int factorial (int n) {
  // n为1结果为1
  if (n == 1)
    return 1;
  // 否则进行递归计算
  return n * factorial (n - 1);
}
```

70. C 语言标准规定，注释被替换为空白符，连续多个空白符则合并为一个空白符。按此规定，写出代码3.2经过词法分析后得到的单词二元组序列。

代码 3.2 C 语言计算阶乘

```
int factorial (int/*形参*/n) {
  if (n =/*是否为1*/= 1)
  return 1;
  return n * fact/*递归调用*/orial(n - 1);
}
```

71. 在词法分析中，如果注释被直接删除，写出代码3.2经过词法分析后得到的单词二元组序列。
72. 写出代码3.3经过词法分析后得到的单词二元组序列。

代码 3.3 计算最大值

```
int max(int m, int n) {
  return m > n ? m : n;
}
int mian() {
  int inputX;
  scanf("%d", &inputX);
  printf("max = %d\r\n", max(inputX, -1));
}
```

山东大学 郑艳伟

第 4 章 语法分析

第 4 章 练习

1. 语法分析按分析的方向分为哪两大类？各有哪些典型的分析方法？
2. LL(1) 分析法中，各字母和数字的含义是什么？
3. 语法分析中，什么是回溯试探法？其缺点是什么？
4. 进行自上而下分析前，需要对文法进行预处理，确保文法符合哪些要求？
5. 左递归和左公因子在自上而下分析中的危害是什么？
6. 能进行自上而下 LL(1) 分析的条件是什么？
7. 写出显示左递归消除的公式。
8. 写出 LL(1) 分析中 $First(\alpha)$、$First(A)$ 和 $Follow(A)$ 的定义，其中 α 是任意产生式的右部（候选式），A 为任意非终结符。
9. 文法 $G[S]$，产生式如下所示，消除其左递归。

   $S \to Aa|b$　　$A \to Bb|c$　　$B \to Cc|d$　　$C \to Sd|a$

10. 文法 $G[S]$，产生式如下所示，消除其左递归。

    $S \to Aa|b$　　$A \to Bc|d$　　$B \to Se|Af|h$

11. 文法 $G[S]$，产生式如下所示，消除其左递归。

    $S \to Aa$　　$S \to b$　　$A \to cS$　　$A \to Sd$

12. C 风格 if 语句文法，$G[S] : S \to if\ (B)\ S \mid if\ (B)\ S\ else\ S$，提取左公因子。
13. C 风格 while 语句文法，$G[S] : S \to while\ (B)\ S \mid while\ (B)\ \{S\}$，提取左公因子。
14. Matlab 风格 for 语句文法，$G[S] : S \to for\ E : E\ do\ S \mid for\ E : E : E\ do\ S$，提取左公因子。
15. 文法 $G[S]$，产生式如下。

    $S \to ABCDE$　　$A \to a|\varepsilon$　　$B \to Bb|\varepsilon$

    $C \to Cc|\varepsilon$　　$D \to DE|\varepsilon$　　$E \to eS|\varepsilon$

    (1) 构造所有非终结符的 $First$ 集合；

    (2) 构造所有非终结符的 $Follow$ 集合。

16. 文法 $G[S]$，产生式如下。

    $S \to ABCDE$　　$A \to Aa|\varepsilon$　　$B \to Bb|\varepsilon$

    $C \to DE$　　$D \to DE|\varepsilon$　　$E \to eS|\varepsilon$

    (1) 构造所有非终结符的 $First$ 集合；

    (2) 构造所有非终结符的 $Follow$ 集合。

17. 文法 $G[S]$，产生式如下。

    $S \to ABCDE$　　$A \to BCDE$　　$B \to CDE$

    $C \to DE$　　$D \to DE|\varepsilon$　　$E \to eS|\varepsilon$

    (1) 构造所有非终结符的 $First$ 集合；

    (2) 构造所有非终结符的 $Follow$ 集合。

山东大学 郑艳伟

18. 文法 $G[S]$，产生式如下。

$$S \to aBc \qquad S \to bAB \qquad A \to aAb$$
$$A \to b \qquad B \to b \qquad B \to \varepsilon$$

(1) 构造所有非终结符的 $First$ 集合；
(2) 构造所有非终结符的 $Follow$ 集合；
(3) 构造 LL(1) 分析表；
(4) 写出 $baabbb$ 的识别过程；
(5) 写出递归下降分析程序。

19. C 风格声明语句文法 $G[L]$，产生式如下，其中 id 为变量名。

$$L \to L; D \qquad L \to D \qquad D \to T\ id$$
$$D \to D, id \qquad T \to int \qquad T \to double$$

(1) 消除文法产生式的左递归；
(2) 构造所有非终结符的 $First$ 集合；
(3) 构造所有非终结符的 $Follow$ 集合；
(4) 构造 LL(1) 分析表；
(5) 写出 $int\ i; double\ x, y$ 的识别过程；
(6) 写出递归下降分析程序。

20. Pascal 风格声明语句文法 $G[L]$，产生式如下，其中 id 为变量名。

$$L \to L; D \qquad L \to D \qquad D \to id : T$$
$$D \to id, D \qquad T \to int \qquad T \to real$$

(1) 消除文法的左递归和左公因子；
(2) 构造所有非终结符的 $First$ 集合；
(3) 构造所有非终结符的 $Follow$ 集合；
(4) 构造 LL(1) 分析表；
(5) 写出 $i : int; x, y : real$ 的识别过程；
(6) 写出递归下降分析程序。

21. 带数组引用的表达式文法 $G[E]$，产生式如下。

$$E \to E + T \qquad E \to T \qquad T \to T * F \qquad T \to F$$
$$F \to id \qquad F \to num \qquad F \to Elist] \qquad F \to (E)$$
$$Elist \to id[E \qquad Elist \to Elist, E$$

(1) 消除文法的左递归和左公因子；
(2) 构造所有非终结符的 $First$ 集合；
(3) 构造所有非终结符的 $Follow$ 集合；
(4) 构造 LL(1) 分析表；
(5) 写出 $a[i, j] = a[i, j-1] * 2$ 的识别过程；
(6) 写出递归下降分析程序。

22. 正规式表达式文法 $G[E]$，产生式如下，其中 “|” 为正规式中的或运算，而不是文法的元语言符合。

山东大学 郑艳伟

$E \to E|T$ $\quad E \to T$ $\quad T \to TP$ $\quad T \to P$

$P \to P*$ $\quad P \to F$ $\quad F \to (E)$ $\quad F \to i$

(1) 消除文法的左递归和左公因子；
(2) 构造所有非终结符的 $First$ 集合；
(3) 构造所有非终结符的 $Follow$ 集合；
(4) 构造 LL(1) 分析表；
(5) 写出 $(a|b)*aa$ 的识别过程；
(6) 写出递归下降分析程序。

23. 简化的 if 语句文法 $G[S]$，产生式如下。

$S \to iCtSE$ $\quad S \to s$ $\quad E \to eS$

$E \to \varepsilon$ $\quad C \to c$

(1) 构造所有非终结符的 $First$ 集合；
(2) 构造所有非终结符的 $Follow$ 集合；
(3) 构造 LL(1) 分析表，如果发生冲突，按如下规则干预：如果一个候选式是另一个候选式的前缀，则较长的产生式优先；
(4) 写出 $ictictses$ 的识别过程；
(5) 写出递归下降分析程序。

24. Matlab 风格 for 语句文法 $G[S]$，产生式如下，其中 i 表示任意变量或常量（假设词法分析时 -1 被识别为一个常量，而不是运算符 $-$ 和常量 1）。

$S \to for\ E : E\ do\ S$ $\quad S \to for\ E : E : E\ do\ S$ $\quad S \to s$

$E \to E + T$ $\quad E \to T$ $\quad T \to i$

(1) 消除文法的左递归和左公因子；
(2) 构造所有非终结符的 $First$ 集合；
(3) 构造所有非终结符的 $Follow$ 集合；
(4) 构造 LL(1) 分析表；
(5) 写出 $for\ 100 : -1 : a + b\ do\ for\ 1 : 10\ do\ s$ 的识别过程；
(6) 写出递归下降分析程序。

25. 过程调用文法 $G[E]$，产生式如下，其中 i 表示任意过程名、变量名或常量。

$E \to id(Elist$ $\quad E \to id()$ $\quad Elist \to E, Elist$ $\quad Elist \to E)$

$E \to E + T$ $\quad E \to T$ $\quad T \to id$

(1) 消除文法的左递归和左公因子；
(2) 构造所有非终结符的 $First$ 集合；
(3) 构造所有非终结符的 $Follow$ 集合；
(4) 构造 LL(1) 分析表；
(5) 写出 $max(x, y + z)$ 的识别过程；
(6) 写出递归下降分析程序。

26. 算符之间的优先关系有哪些？

27. 算符优先关系是否为偏序关系？为什么？

山东大学 郑艳伟

28. 什么是算符文法？
29. 什么是算符优先文法？
30. 什么是素短语？
31. 简述最左素短语定理。
32. 文法 $G[S]$ 的产生式如下，试判断该文法是否为算符文法。如果不是，将其改造为算符文法。

    $S \to AB$　　$A \to Aa|b$　　$B \to aB|b$

33. 文法 $G[S]$ 的产生式如下，试判断该文法是否为算符文法。如果不是，将其改造为算符文法。

    $S \to ABC$　　$A \to Ao|s$　　$B \to Bo|\varepsilon$　　$C \to oC|s$

34. 文法 $G[S]$ 的产生式如下，试判断该文法是否为算符文法。如果不是，将其改造为算符文法。

    $S \to ABC$　　$A \to Ao|s$　　$B \to B0|\varepsilon$　　$C \to oC|s$

35. 文法 $G[S]$ 的产生式如下，试判断以下算符的优先关系：θ_1 和 θ_1、θ_1 和 θ_2、θ_1 和 θ_3、θ_2 和 θ_3、θ_3 和 θ_3、θ_3 和 θ_2、a 和 θ_1、a 和 θ_2、a 和 θ_3、θ_1 和 a、θ_2 和 a、θ_3 和 a、a 和 a。

    $S \to S\theta_1 A$　　$S \to A$　　$A \to A\theta_2 B$　　$A \to B$

    $B \to C\theta_3 B$　　$B \to C$　　$C \to (S)$　　$C \to a$

36. 写出算符优先分析法中 $FirstV_T(A)$ 和 $LastV_T(A)$ 的定义，其中 A 是任意非终结符。
37. 什么是算符优先函数？
38. 是否所有算符优先表都存在算符优先函数？为什么？
39. 算符优先表的算符优先函数是否唯一？为什么？
40. 优先函数相对于优先关系表，有何优缺点？
41. 文法 $G[E]$，产生式如下，其中 i 表示任意变量或常量。

    $E \to E + T$　　$E \to T$　　$T \to T * F$　　$T \to F$

    $F \to (E)$　　$F \to i$

    (1) 构造所有非终结符的 $FirstV_T$ 集合；
    (2) 构造所有非终结符的 $LastV_T$ 集合；
    (3) 计算文法终结符的优先关系；
    (4) 计算文法终结符的优先函数；
    (5) 写出 $(a + b) * 2$ 的识别过程。

42. 文法 $G[S]$，产生式如下。

    $S \to a$　　$S \to \hat{}$　　$S \to (T)$　　$T \to T, S$

    $T \to S$

    (1) 构造所有非终结符的 $FirstV_T$ 集合；
    (2) 构造所有非终结符的 $LastV_T$ 集合；
    (3) 计算文法终结符的优先关系；
    (4) 计算文法终结符的优先函数；
    (5) 写出 $(a, \hat{}, (a, a))$ 的识别过程。

43. 文法 $G[E]$，产生式如下，其中 i 表示任意变量或常量。

    $E \to (A, E, E)$　　$E \to i$　　$A \to +$　　$A \to *$

山东大学 郑艳伟

(1) 构造所有非终结符的 $FirstV_T$ 集合；

(2) 构造所有非终结符的 $LastV_T$ 集合；

(3) 计算文法终结符的优先关系；

(4) 计算文法终结符的优先函数；

(5) 写出 $(*,(+,a,b),(+,a,b))$ 的识别过程。

44. LR(k) 分析法中，各字母和数字的含义是什么？

45. LR 分析方法族包括哪些方法？简要说明这些方法的特点。

46. LR 分析法的特点是什么？

47. 简述 LR(0) 分析的基本思想。

48. 什么是规范句型？什么是活前缀？

49. 什么是 LR(k) 项目？什么是 LR(1) 有效项目？

50. SLR(1) 分析法和 LR(1) 分析法都向右查看一个符号，它们之间的本质区别是什么？

51. 文法 $G[S]=(\{S,A\},\{0,1\},P,S)$，产生式如下。

$S \to AS$ $\quad$ $S \to 1$ $\quad$ $A \to SA$ $\quad$ $A \to 0$

(1) 拓广文法；

(2) 构造拓广文法的 LR(0) 项目集规范族；

(3) 计算拓广文法非终结符的 $Follow$ 集合；

(4) 构造拓广文法的 SLR(1) 分析表。

52. 证明下面的文法 $G[S]$ 是 SLR(1) 文法，但不是 LR(0) 文法。

$S \to A$ $\quad$ $A \to Ab$ $\quad$ $A \to bBa$

$B \to aAc$ $\quad$ $B \to aAb$ $\quad$ $B \to a$

53. 证明下面的文法 $G[S]$ 是 LR(1) 文法，但不是 SLR(1) 文法。

$S \to A0A1$ $\quad$ $S \to B1B0$ $\quad$ $A \to \varepsilon$ $\quad$ $B \to \varepsilon$

54. C 风格声明语句文法 $G[L]$，产生式如下，其中 id 为变量名。

$L \to L;D$ $\quad$ $L \to D$ $\quad$ $D \to T\ id$

$D \to D,id$ $\quad$ $T \to int$ $\quad$ $T \to double$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 构造拓广文法的 LR(1) 分析表；

(4) 写出句子 $int\ a,\ b;\ double\ c$ 的识别过程。

55. Pascal 风格声明语句文法 $G[L]$，产生式如下，其中 id 为变量名。

$L \to L;D$ $\quad$ $L \to D$ $\quad$ $D \to id:T$

$D \to id,D$ $\quad$ $T \to int$ $\quad$ $T \to real$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 构造拓广文法的 LR(1) 分析表；

(4) 写出句子 $a,\ b:\ int;\ c:\ real$ 的识别过程。

山东大学 郑艳伟

56. 过程调用文法 $G[E]$，产生式如下，其中 id 为过程名、变量名或常量。

$$E \to id(L \qquad L \to E, L \qquad L \to E) \qquad E \to id$$

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 构造拓广文法的 LR(1) 分析表；
(4) 写出句子 $s(x, y)$ 的识别过程。

57. 识别正规式的文法 $G[E]$，产生式如下，其中 i 表示任意字符。

$$E \to E|E \qquad E \to EE \qquad E \to E*$$
$$E \to (E) \qquad E \to i$$

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 构造拓广文法的 LR(1) 分析表，如果产生冲突，按如下规则干预：非、与、或优先级由高到低，非运算右结合，与、或运算左结合；
(4) 写出句子 $(a|b) * a$ 的识别过程。

58. 简化的 if 语句文法 $G[S]$，产生式如下。

$$S \to iCtSE \qquad S \to s \qquad E \to eS$$
$$E \to \varepsilon \qquad C \to c$$

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 构造拓广文法的 LR(1) 分析表，如果产生冲突，按如下规则干预：如果一个候选式是另一个候选式的前缀，则较长的产生式优先；
(4) 写出句子 $ictictses$ 的识别过程。

59. Matlab 风格 for 语句文法 $G[S]$，产生式如下，其中 i 表示任意变量或常量（假设词法分析时 -1 被识别为一个常量，而不是运算符 $-$ 和常量 1）。

$$S \to for\ E : E\ do\ S \qquad S \to for\ E : E : E\ do\ S \qquad S \to s$$
$$E \to E + T \qquad E \to T \qquad T \to i$$

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 构造拓广文法的 LR(1) 分析表，如果产生冲突，按如下规则干预：如果一个子串是另一个子串的前缀，则较长的产生式优先；
(4) 写出句子 $for\ 100 : -1 : a + b\ do\ for\ 1 : 10\ do\ s$ 的识别过程。

60. 什么是同心集？

61. 合并同心集会不会产生新的移进-归约冲突？为什么？

62. 乘幂运算文法 $G[E]$，产生式如下。

$$E \to E * E \qquad E \to E\hat{}E \qquad E \to (E) \qquad E \to i$$

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 如有可能，合并同心集；

山东大学 郑艳伟

(4) 构造 LALR(1) 分析表，如果产生冲突，按如下规则干预：幂运算优先级高于乘法运算，乘法左结合，幂运算右结合；

(5) 写出句子 $i*i\hat{}i\hat{}i$ 的识别过程。

63. 负运算和减法构成的系统，其文法 $G[E]$，产生式如下。

$E \to E - E$ $\quad$ $E \to -E$ $\quad$ $E \to (E)$ $\quad$ $E \to i$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 如有可能，合并同心集；

(4) 构造 LALR(1) 分析表，如果产生冲突，按如下规则干预：负运算优先级高于减法运算，减法左结合，负运算右结合；

(5) 写出句子 $-(i---i)$ 的识别过程。

64. 文法 $G[S]$，产生式如下。

$S \to AA$ $\quad$ $A \to oA$ $\quad$ $A \to s$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 如有可能，合并同心集；

(4) 构造 LALR(1) 分析表；

(5) 写出句子 sos 的识别过程。

65. 文法 $G[S]$，产生式如下。

$S \to xXu$ $\quad$ $S \to yYu$ $\quad$ $S \to yXv$ $\quad$ $S \to xYv$

$X \to w$ $\quad$ $Y \to w$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 找出同心集；

(4) 同心集是否可以合并？为什么？

66. 语句序列文法 $G[S]$，产生式如下，其中 s 表示任意语句，M 是语句翻译时为了插入动作引入的空符产生式。

$S \to \{L\}$ $\quad$ $L \to S$ $\quad$ $L \to LMS$

$S \to s;$ $\quad$ $M \to \varepsilon$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 构造拓广文法的 LR(1) 分析表；

(4) 写出句子 $\{sss\}$ 的识别过程。

67. Pascal 风格过程内声明语句文法 $G[S]$，产生式如下，其中 id 表示变量名，分号左结合，M 是语句翻译时为了插入动作引入的空符产生式。

$P \to MD$ $\quad$ $M \to \varepsilon$ $\quad$ $D \to D;D$

$D \to id\ L$ $\quad$ $L \to ,id\ L$ $\quad$ $L \to :T$

$T \to int$ $\quad$ $T \to real$

山东大学 郑艳伟

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 构造拓广文法的 LR(1) 分析表；
(4) 写出句子 $i, j : int; a : real$ 的识别过程。

68. Pascal 风格过程内数组声明语句文法 $G[V]$，产生式如下，其中 id 表示变量名，num 是常量，arr 是表示数组的关键字。

$V \to id\ L$　　$L \to, id\ L$　　$L \to : T$
$T \to int$　　$T \to real$　　$A \to arr[num : num$
$A \to A, num : num$　　$T \to A]\ of\ T$

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 构造拓广文法的 LR(1) 分析表；
(4) 写出句子 $a : arr\ [0 : 2,\ 3 : 7]\ of\ int$ 的识别过程。

69. C 风格过程定义语句文法 $G[F]$，产生式如下，其中 id 表示过程名或变量名，N 是语句翻译时为了插入动作引入的空符产生式，s 是除过程定义外的任意程序语句。

$F \to HM)NS$　　$H \to T\ id($　　$N \to \varepsilon$
$M \to V$　　$M \to M, V$　　$V \to T\ id$
$S \to s$　　$T \to void$　　$T \to int$

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 构造拓广文法的 LR(1) 分析表；
(4) 写出句子 $int\ sum(int\ x,\ int\ y)$ 的识别过程。

70. C 风格结构体定义语句文法 $G[S]$，产生式如下，其中 id 表示结构体名或变量名。

$S \to RM\};$　　$S \to RM\}\ id;$　　$R \to struct\ id\{$
$M \to V$　　$M \to MV$　　$V \to T\ id;$
$T \to int$　　$T \to real$　　$T \to struct\ id$

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 构造拓广文法的 LR(1) 分析表；
(4) 写出句子 $struct\ PT\{int\ x,\ int\ y;\ struct\ RGB\ color;\};$ 的识别过程。

71. 控制语句文法 $G[S]$，产生式如下，其中 s 表示任意语句，b 是布尔表达式，M 和 N 是语句翻译时为了插入动作引入的空符产生式。

$S \to if\ (B)\ MSN\ else\ MS$　　$S \to if\ (B)\ MS$　　$S \to while\ (MB)\ MS$
$S \to s$　　$B \to b$　　$M \to \varepsilon$
$N \to \varepsilon$

(1) 拓广文法；
(2) 构造拓广文法的 LR(1) 项目集规范族；
(3) 构造拓广文法的 LR(1) 分析表；

山东大学 郑艳伟

(4) 写出句子 $while\ (b)\ if\ (b)\ s\ else\ s$ 的识别过程。

72. Matlab 风格 for 语句文法 $G[S]$，产生式如下，其中 s 表示任意语句，e 是算术表达式，M 是语句翻译时为了插入动作引入的空符产生式，id 为变量名。

$S \to for(H)S$ $\quad H \to JME$ $\quad H \to JME : E$

$J \to id = E :$ $\quad S \to s$ $\quad M \to \varepsilon$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 构造拓广文法的 LR(1) 分析表；

(4) 写出句子 $for\ (i = 100 : -1 : 0)\ for\ (j = 1 : 10)\ s$ 的识别过程，其中每个常量都与 e 匹配（假设词法分析时 -1 被识别为一个常量，而不是运算符 $-$ 和常量 1）。

73. 标号–goto 语句文法 $G[S]$，产生式如下，其中 s 表示任意语句，id 表示标号名称。

$S \to label\ S$ $\quad label \to id :$ $\quad S \to goto\ id;$

$S \to s$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 构造拓广文法的 LR(1) 分析表；

(4) 写出句子 $goto\ a;\ a:\ s\ goto\ a;$ 的识别过程。

74. Switch 语句文法 $G[S]$，产生式如下，其中 s 表示任意语句，e 表示算术表达式，c 表示常量。

$S \to W\{UX\}$ $\quad W \to switch(E)$ $\quad E \to e$

$U \to CS$ $\quad C \to case\ c :$ $\quad C \to U\ case\ c :$

$X \to default : MS$ $\quad X \to \varepsilon$ $\quad M \to \varepsilon$

$S \to s$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 构造拓广文法的 LR(1) 分析表；

(4) 写出句子 $switch(e)\ \{case\ 0:\ s\ case\ 1:\ s\ default:\ s\}$ 的识别过程。

75. Lisp 风格表达式文法 $G[E]$，产生式如下，其中 i 表示任意变量或常量。

$E \to (A, E, E)$ $\quad E \to i$ $\quad A \to +$ $\quad A \to *$

(1) 拓广文法；

(2) 构造拓广文法的 LR(1) 项目集规范族；

(3) 构造拓广文法的 LR(1) 分析表；

(4) 写出句子 $(*, (+, a, b), (+, a, b))$ 的识别过程。

山东大学 郑艳伟

第 5 章 符号表

第 5 章 练习

1. 什么是符号表？
2. 符号表的作用是什么？
3. 编译期间，符号表有哪些操作？
4. 列举说明简单变量的符号表项内容。
5. 列举说明数组的符号表项内容。
6. 列举说明结构体的符号表项内容。
7. 列举说明过程的符号表项内容。
8. 列举说明标号的符号表项内容。
9. 试说明 Pascal 风格嵌套定义过程的变量作用域规则。
10. 试说明 Pascal 风格嵌套定义过程的符号表结构组织方式。
11. 试说明 Pascal 风格嵌套定义过程的符号表栈的工作过程。
12. 符号表的内容组织有哪些方式？
13. 试说明符号表的表格组织方式。
14. 试说明符号表的表记录组织方式。
15. 试说明符号表的面向对象组织方式。
16. 试说明整数的大尾编码和小尾编码，并写出 0AB01h 的两种 4 字节编码形式。
17. 字符串的存储方式有哪些？写出字符串 “abcd” 使用不同存储方式时的字节序列。
18. 写出以下代码的完整符号表。

```
struct STRU {
  int a;
  float b[5, 10];
  double c;
};
int Fun(STRU s, int len){
  STRU temp;
  int i, j;
}
```

19. 写出下表中变量的表记录存储内容。

名字	数据类型	类别	宽度	偏移量
x	int	形参变量	4	0
y	int	形参变量	4	4
i	int	普通变量	4	0
j	int	普通变量	4	4
u	double	普通变量	8	8

山东大学 郑艳伟

v	double	普通变量	8	16
arr	array(2, 3, 4, int)	普通变量	96	24

20. 对变量 x、y、i、j、u、v、arr，写出变量的线性组织形式。
21. 对变量 x、y、i、j、u、v、arr，按名字字符串比较大小，写出变量的二叉树组织形式。
22. 对变量 x、y、i、j、u、v、arr，按名字字符串比较大小，写出变量的平衡二叉树组织形式。
23. 对变量 x、y、i、j、u、v、arr，按名字 ASCII 码值累加转换为数字表示，按直接定址法构造存储映射函数，线性探测再散列解决冲突，写出变量的哈希表组织形式。
24. 对变量 x、y、i、j、u、v、arr，按名字 ASCII 码值累加转换为数字表示，按直接定址法构造存储映射函数，平方探测再散列解决冲突，写出变量的哈希表组织形式。
25. 对变量 x、y、i、j、u、v、arr，按名字 ASCII 码值累加转换为数字表示，按直接定址法构造存储映射函数，链地址法解决冲突，写出下表中变量的哈希表组织形式。
26. 对变量 prod、factorial、fact、num1、num2、var、visible，写出变量的线性组织形式。
27. 对变量 prod、factorial、fact、num1、num2、var、visible，按名字字符串比较大小，写出变量的二叉树组织形式。
28. 对变量 prod、factorial、fact、num1、num2、var、visible，按名字字符串比较大小，写出变量的平衡二叉树组织形式。
29. 对变量 prod、factorial、fact、num1、num2、var、visible，按名字 ASCII 码值累加转换为数字表示，按直接定址法构造存储映射函数，线性探测再散列解决冲突，写出变量的哈希表组织形式。
30. 对变量 prod、factorial、fact、num1、num2、var、visible，按名字 ASCII 码值累加转换为数字表示，按直接定址法构造存储映射函数，平方探测再散列解决冲突，写出变量的哈希表组织形式。
31. 对变量 prod、factorial、fact、num1、num2、var、visible，按名字 ASCII 码值累加转换为数字表示，按直接定址法构造存储映射函数，链地址法解决冲突，写出下表中变量的哈希表组织形式。

山东大学 郑艳伟

第 6 章 运行时存储空间组织

第 6 章 练习

1. 简述运行时不同存储区域的访问方式。
2. 什么是活动记录？活动记录一定是栈帧结构吗？
3. 画图表示过程的活动记录，并说明各部分存放的内容。
4. 有哪些运行时存储分配策略？何时使用何种存储策略？
5. 高级程序语言的传参方式有哪些？具体是怎么做的？
6. 过程调用规范主要有哪些？
7. 简述 std call 过程调用规范。
8. 简述 C call 过程调用规范。
9. 简述 x64 过程调用规范。
10. 简述 ARM 过程调用规范。
11. 过程 factorial() 传入一个整型参数 n，通过递归调用计算 n!。main() 过程调用 factorial(5)，使用 std call 过程调用规范，编写汇编代码。
12. 过程 factorial() 传入一个整型参数 n，通过递归调用计算 n!。main() 过程调用 factorial(5)，使用 C call 过程调用规范，编写汇编代码。
13. 过程 factorial() 传入一个整型参数 n，通过递归调用计算 n!。main() 过程调用 factorial(5)，使用 x64 过程调用规范，编写汇编代码。
14. 过程 swap() 传入两个地址参数，执行后交换两个地址中保存的内容。main() 过程使用 push 指令压入数字 0 和 1，然后把这两个数字的地址传递给过程 swap() 进行调用。使用 std call 过程调用规范，编写汇编代码。
15. 过程 swap() 传入两个地址参数，执行后交换两个地址中保存的内容。main() 过程使用 push 指令压入数字 0 和 1，然后把这两个数字的地址传递给过程 swap() 进行调用。使用 C call 过程调用规范，编写汇编代码。
16. 过程 swap() 传入两个地址参数，执行后交换两个地址中保存的内容。main() 过程使用 push 指令压入数字 0 和 1，然后把这两个数字的地址传递给过程 swap() 进行调用。使用 x64 过程调用规范，编写汇编代码。
17. 过程 factorial() 传入一个整型参数 n，通过递归调用计算 n!。main() 过程调用 factorial(3)，使用 std call 过程调用规范，画图表示每调用一个过程时的活动记录。
18. 过程 factorial() 传入一个整型参数 n，通过递归调用计算 n!。main() 过程调用 factorial(3)，使用 C call 过程调用规范，画图表示每调用一个过程时的活动记录。
19. 过程 factorial() 传入一个整型参数 n，通过递归调用计算 n!。main() 过程调用 factorial(3)，使用 x64 过程调用规范，画图表示每调用一个过程时的活动记录。
20. 将结构体对象作为过程参数，通过传值进行参数传递，试设计参数传递策略。
21. 根据以下代码，写出运行时进入函数 Fun 后的整个栈帧。

```
struct STRU {
  int a;
```

山东大学 郑艳伟

```
    float b[5, 10];
    double c;
};
int Fun(STRU s, int len){
    STRU temp;
    int i, j;
}
int main() {
    STRU a;
    ...
    Fun(a, 10);
    ...
    return 0;
}
```

22. 根据以下代码，写出通过 EBP 寄存器访问如下变量的访问地址：len、s.a、s.b[2, 5]、s.c、temp.a、temp.b[2, 5]、temp.c、i、j。

```
struct STRU {
    int a;
    float b[5, 10];
    double c;
};
int Fun(STRU s, int len){
    STRU temp;
    int i, j;
}
int main() {
    STRU a;
    ...
    Fun(a, 10);
    ...
    return 0;
}
```

23. 根据以下代码，写出运行时进入函数 Fun 后的整个栈帧。

```
struct STRU {
    int a;
    float b[5, 10];
    double c;
};
int Fun(STRU &s, int len){
    STRU temp;
    int i, j;
}
int main() {
```

山东大学 郑艳伟

```
    STRU a;
     ...
    Fun(a, 10);
     ...
    return 0;
  }
```

24. 根据以下代码，写出通过 EBP 寄存器访问如下变量的访问地址：len、s.a、s.b[2, 5]、s.c、temp.a、temp.b[2, 5]、temp.c、i、j。

```
  struct STRU {
    int a;
    float b[5, 10];
    double c;
  };
  int Fun(STRU &s, int len){
    STRU temp;
    int i, j;
  }
  int main() {
    STRU a;
     ...
    Fun(a, 10);
     ...
    return 0;
  }
```

25. 使用 x86 汇编设计一个控制台程序，该程序调用 C 运行时库的 printf 函数，将所有通用寄存器的内容打印到屏幕。
26. 简述跨文件调用的方法。
27. 简述封装库和调用库的方法。
28. 简述静态链的构造方法。
29. 简述通过静态链访问外层变量的方法。
30. 什么是嵌套层次显示表？
31. 简述通过 Display 表访问变量的方法。
32. 根据如下 Pascal 程序，当递归地第二次进入函数 factorial 后，Display 的内容是什么？此时整个运行时栈帧的内容是什么？

```
  program Pgrm(input, output)
    var result : integer;
    function factorial(n : integer) : integer
    begin
      if n <= 0 then factorial := 1
      else factorial := n * factorial(n - 1);
    end;
  begin
```

北京大学 郑艳伟

```
    result = factorial(10);
    ...
end.
```

33. 堆式存储分配有哪些方式？具体怎么做的？
34. 堆式存储回收有哪些方式？具体怎么做的？
35. 如下 C++ 代码，通过调用 C 运行时库的 malloc 和 free 函数实现堆内存的申请和释放，使用汇编代码实现它。

```
int *p = new int[20];
...
delete p;
```

山东大学 郑艳伟

第 7 章 语法制导翻译与中间代码生成

第 7 章 练习

1. 静态语义检查的任务有哪些？
2. 简述四元式有哪些属性。
3. 什么是属性文法？
4. 什么是综合属性？什么是继承属性？
5. 终结符可以有哪类属性？如何计算？
6. 开始符号的继承属性是如何计算的？
7. 什么是语法制导翻译？
8. 什么是 S-属性文法？什么是 L-属性文法？
9. 什么是属性计算的依赖图？它是如何构建的？
10. 下列文法 $G[E]$ 对整型常数和实型常数施加加乘运算。将两个整型数相加得到整型结果，否则结果为实型。试给出确定每个子表达式结果数据类型的属性文法。

    $E \to E + T$　　$E \to T$　　$T \to T * F$　　$T \to F$

    $F \to (E)$　　$F \to num.num$　　$F \to num$

11. 下列文法 $G[S]$ 生成一个二进制数。设计综合属性 val，当识别出一个二进制数后，$S.val$ 为这个二进制数对应的十进制数值。如输入 101.01 时，$S.val = 5.25$。

    $S \to L.R$　　$S \to L$　　$L \to LB$

    $L \to B$　　$R \to RB$　　$R \to B$

    $B \to 0$　　$B \to 1$

12. 根据构建抽象语法树的 S–属性文法，构造句子 $(a + b) * (a + b)$ 的抽象语法树。
13. 根据 NFA 箭弧单符化的 S–属性文法，构造句子 $(a|b)^*(aa|bb)$ 的单符化 NFA。
14. 修改如下台式计算器的属性文法，消除左递归，并能进行自上而下计算。

序号	产生式	语义规则
1	$L \to E$	$print(E.value)$;
2	$E \to E_1 + E_2$	$E.value = E_1.value + E_2.value$;
3	$E \to E_1 * E_2$	$E.value = E_1.value * E_2.value$;
4	$E \to (E_1)$	$E.value = E_1.value$;
5	$E \to num$	$E.value = num.lexvalue$;

15. 翻译 Pascal 风格声明语句：a, b : integer; x, y : real
16. 翻译 Pascal 风格声明语句：arr1, arr2 : array[1..5, 2..7] of real; a, b : integer
17. 翻译 C 风格声明语句：int a, b; double x, y;
18. 翻译 C 风格声明语句：int a[2, 4]; double x, y;
19. 翻译如下 C 风格声明语句。

```
int sum(int x, y;) {
  int sum;
}
```

山东大学 郑艳伟

20. 翻译如下 C 风格声明语句。

```
struct STRU {
  int a;
  float b[5, 10];
  double c;
};
int Fun(STRU s; int u, v;) {
  STRU temp;
  int i, j;
}
```

21. 翻译如下 C 风格表达式赋值语句：delt = b * b - 4 * a * c;
22. 翻译如下 C 风格表达式赋值语句：a[i, j] = a[i, 2 * j] * 2;
23. 翻译如下 C 风格表达式赋值语句，其中 s 和 temp 分别为如下代码 Fun 函数的形参和局部变量：temp.b[i, j] = s.b[i, j];

```
struct STRU {
  int a;
  float b[5, 10];
  double c;
};
int Fun(STRU s; int u, v;) {
  STRU temp;
  int i, j;
}
```

24. 翻译如下 C 风格表达式赋值语句：isMin = x <= y && x <= z;
25. 翻译如下 C 风格表达式赋值语句：p = &x; y = *p + x;
26. C 语言中自增和自减表达式的产生式如下，其中 id 为变量名。根据 C 语言语义规则，设计其翻译模式。

$$E \to id++ \qquad E \to ++id \qquad E \to id-- \qquad E \to --id$$

27. C 语言中复合赋值运算符的产生式如下，其中 id 为变量名，num 为常量。根据 C 语言语义规则，设计其翻译模式。

$$E \to id+=E \qquad E \to id*=E \qquad E \to E+E \qquad E \to E*E$$
$$E \to (E) \qquad E \to id \qquad E \to num$$

28. 设计自增和自减表达式的翻译模式，结合表达式与赋值语句，翻译如下句子：a = (++b) + (b−−);
29. 设计复合赋值运算的翻译模式，结合表达式与赋值语句，翻译如下句子：a += b * c;
30. 思考对于含有同一个变量自增/自减的表达式，如 i+++−−i，为什么不同编译器会得到不同的结果？
31. 翻译句子：if (x < i + j && i > j) i = i − 1; x = i;
32. 翻译句子：if (x < i + j && i > j) i = i − 1; else j = j − 1; x = i;
33. 翻译句子：while (x < i + j && i > j) i = i − 1; x = i;
34. 翻译句子：while (x < i + j && i > j) if (j > 10) i = i − 1; x = i;

山东大学 郑艳伟

35. 翻译句子：while (x < i + j && i > j) if (j > 10) i = i − 1; else i = i + 1; x = i;
36. 翻译句子：if (x < i + j && i > j) while (i > 0) i = i − 1; x = i;
37. 翻译句子：while (x < i + j) while (i > j) i = i − 1; x = i;
38. 翻译句子：for (i = 1; i < 10; i = i + 1;) sum = sum + i; i = 0;
39. 翻译句子：for (i = 1 : 10;) sum = sum + i; i = 0;
40. 翻译句子：for (i = 10 : −1 : 1;) sum = sum + i; i = 0;
41. 翻译句子：for (i = 1 : 2 : n;) {sum = sum + i; n = n + 1;} i = 0;
42. 翻译句子：for (i = 1 : 2 : n;) for (j = 1 : n) x = x + i + j; i = 0;
43. 翻译句子：if (i = j) goto L; if (i = 2 * j) goto L; L: x = y + z; goto L;
44. 翻译句子：switch(month){case 2 : day = 28; case 4 : day = 30; case 6: day = 30; case 9: day = 30; case 11: day = 30; default : day = 31; } day = day * 4;
45. 翻译句子，其中 “%”、“=” 和 ‘==” 分别是求余、赋值和关系等运算符：x = n % 2 == 0 ? y + z : y − z;
46. 翻译句子：while (a <= i <= b < j) {i = i + 1; a = a + 2;} i = a + b;
47. 翻译句子：for (i = 1 : 10;) {sum = sum + i; if (sum > 10) break; } i = 0;
48. 翻译句子：x = a > b && a < c ? a : c;
49. 翻译句子：if (a > b > c > d) x = a; y = d;
50. 翻译句子：if (a < b = c > d) x = a + b; y = c + d;
51. 翻译句子：z = a > b ? sum(a * 2, b) : sum(a, b * 2);
52. 什么是类型检查？什么是良类型系统？
53. 以 C 语言为例，写出类型表达式。
54. 写出 C 语言算术运算类型检查的翻译模式。
55. 写出 C 语言关系运算类型检查的翻译模式。
56. 写出 C 语言逻辑运算类型检查的翻译模式。
57. 写出 C 语言数组引用类型检查的翻译模式。
58. 写出 C 语言结构体引用类型检查的翻译模式。
59. 翻译句子，其中 x、y 为实型变量，i、j 为整型变量：x = (y + i) * j;
60. 翻译句子，其中 x、y 为实型变量，i、j 为整型变量：i = (int)(x + y) * j;
61. 翻译句子，其中 x、y 为实型变量，i、j 为整型变量：i = (int)((x + y) * j);
62. 翻译句子，其中 x、y 为实型变量，i、j 为整型变量：x = (real)(i + j) * y;
63. 如果系统只支持常量字符串，则 printf(“abc\r\n”) 可以翻译为四元式 $(param,“abc\backslash r\backslash n”,-,-)$ 和 $(call, printf,-,-)$。为支持这一操作，只需要增加一个语义规则：$E \rightarrow cstr\{E.val = cstr.val\}$，其中 $cstr$ 表示常量字符串，它的属性 val 来自词法分析器。当词法分析器扫描完常量字符串 “abc\r\n”，有 $cstr.val =“abc\backslash r\backslash n”$（词法分析器会将转义字符替换为真实字符）。基于这样的设计，试翻译如下语句：printf(“%d + %d = %d\r\n”, x, y, x + y);
64. 试翻译如下代码。

```
int sum(int x, y;) {
  return x + y;
}
int main()
```

山东大学 郑艳伟

```
{
  int a, b;
  printf (“ Please input two numbers: \r\n”);
  scanf(“%d + %d”, &a, &b);
  printf (“  = %d\r\n”, sum(a, b));
  return 0;
}
```

65. 试翻译如下代码。

```
int factorial (int n;) {
  if (n = 1)
    return 1;
  else
    return n * factorial (n - 1);
}
int main() {
  int n;
  printf (“ Please input a number: \r\n”);
  scanf(“%d’’, &n);
  printf(“%d! = %d\r\n”, n, factorial (n));
  return 0;
}
```

66. 试翻译如下快排代码，其中形参需要修改声明语句的翻译模式，使其接受不指定维度信息的数组；或者也可以将代码中第一个参数修改为指针类型，通过地址偏移访问元素。

```
void swap(int *x; int *y;) {
  int t = *x;
  *x = *y;
  *y = t;
}
void quickSort(int arr []; int start, end;) {
  if (start >= end)
    return;
  int pivot = arr[end];
  int left = start, right = end - 1;
  while (left < right) {
    while (arr[left] < pivot && left < right)
      left = left + 1;
    while (arr[right] >= pivot && left < right)
      right = right - 1;
    swap(&arr[left], &arr[right]);
  }
  if (arr[left] >= arr[end])
    swap(&arr[left], &arr[end]);
  else
```

山东大学 郑艳伟

```
        left = left + 1;
    if (left > 0)
        quickSort(arr, start, left - 1);
    quickSort(arr, left + 1, end);
}
```

67. 本章在计算变量偏移量时，没有考虑字节对齐问题。如果硬件一次读取一个 32 位的双字，那么一个数字跨双字存储，就会导致需要读取两次才能取到这个数。为此，需要将存储对齐到 4 字节的倍数。如一个 byte 类型和一个 int 类型连续存储，byte 类型后需要空 3 字节；两个 byte 类型和一个 int 类型连续存储，两个 byte 类型连续存储，之后空 2 字节；最终目的都是让一个完整数据不跨双字存储。根据这个约定，设计声明语句翻译模式中 $offset$ 的计算方法。
68. 如果要支持形式参数的引用调用，试考虑过程定义、过程调用的语义规则如何设计，符号表如何设计。

山东大学 郑艳伟

第 8 章 中间代码优化

第 8 章 练习

1. 基本块入口语句有哪些？
2. 确定基本块入口语句后，如何划分基本块？
3. 流图构造的规则有哪些？
4. 什么是支配结点和支配结点集？
5. 循环有哪些性质？循环的定义是什么？
6. 什么是回边？
7. 什么是严格支配？什么是直接支配？
8. 什么是支配树？
9. 对如下求和代码：

$100.(proc, evenSum, -, -)$	$104.(j, -, -, 110)$	$108.(=, \$3, -, i)$
$101.(=, 1, -, i)$	$105.(+, sum, i, \$2)$	$109.(j, -, -, 103)$
$102.(=, 0, -, sum)$	$106.(=, \$2, -, sum)$	$110.(ret, sum, -, -)$
$103.(j \leq, i, n, 105)$	$107.(+, i, 1, \$3)$	$111.(endp, evenSum, -, -)$

   (1) 划分基本块；
   (2) 构造流图；

10. 对如下求偶数和代码：

$100.(proc, evenSum, -, -)$	$105.(\%, i, 2, \$1)$	$110.(+, i, 1, \$3)$
$101.(=, 1, -, i)$	$106.(j =, \$1, 0, 108)$	$111.(=, \$3, -, i)$
$102.(=, 0, -, sum)$	$107.(j, -, -, 110)$	$112.(j, -, -, 103)$
$103.(j \leq, i, n, 105)$	$108.(+, sum, i, \$2)$	$113.(ret, sum, -, -)$
$104.(j, -, -, 113)$	$109.(=, \$2, -, sum)$	$114.(endp, evenSum, -, -)$

   (1) 划分基本块；
   (2) 构造流图；

11. 对如下阶乘代码：

$100.(proc, fact, -, -)$	$104.(-, n, 1, \$1)$	$108.(*, n, \$2, \$3)$
$101.(j =, n, 1, 103)$	$105.(param, \$1, -, -)$	$109.(ret, \$3, -, -)$
$102.(j, -, -, 104)$	$106.(call, fact, -, -)$	$110.(endp, fact, -, -)$
$103.(ret, 1, -, -)$	$107.(=, fact, -, \$2)$	

   (1) 划分基本块；
   (2) 构造流图；

12. 对如下代码：

$100.(proc, fun, -, -)$	$109.(=, 1, -, j)$	$118.(+, sum1, j, \$4)$
$101.(=, 0, -, sum1)$	$110.(j \leq, j, i, 115)$	$119.(=, \$4, -, sum1)$
$102.(=, 0, -, sum2)$	$111.(j, -, -, 106)$	$120.(j, -, -, 112)$

山东大学 郑艳伟

103.$(=,1,-,i)$	112.$(+,j,1,\$2)$	121.$(+,sum2,j,\$5)$
104.$(j\leq,i,n,109)$	113.$(=,\$2,-,j)$	122.$(=,\$5,-,sum2)$
105.$(j,-,-,124)$	114.$(j,-,-,110)$	123.$(j,-,-,112)$
106.$(+,i,1,\$1)$	115.$(\%,j,2,\$3)$	124.$(*,sum1,sum2,\$6)$
107.$(=,\$1,-,i)$	116.$(j=,\$3,0,118)$	125.$(ret,\$6,-,-)$
108.$(j,-,-,104)$	117.$(j,-,-,121)$	126.$(endp,fun,-,-)$

(1) 划分基本块；
(2) 构造流图；
(3) 计算各基本块支配结点集；
(4) 识别回边；
(5) 识别循环；
(6) 计算各循环层次；
(7) 构造支配树。

13. 如流图8.1，B_1 为入口基本块，B_7 为出口基本块：

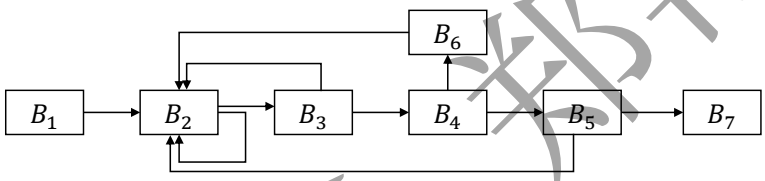


图 8.1 循环流图

(1) 计算各基本块支配结点集；
(2) 识别回边；
(3) 识别循环；
(4) 计算各循环层次；
(5) 构造支配树。

14. 如流图8.2，B_1 为入口基本块，B_8 为出口基本块：

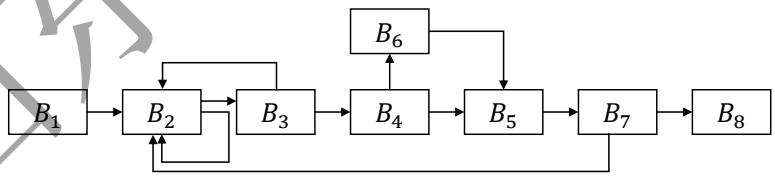


图 8.2 循环流图

(1) 计算各基本块支配结点集；
(2) 识别回边；
(3) 识别循环；
(4) 计算各循环层次；
(5) 构造支配树。

15. 优化的基本原则是什么？
16. 什么是变量的定值和引用？

山东大学 郑艳伟

17. 什么是变量的活跃性？
18. 什么是删除公共子表达式？
19. 什么是复写传播？
20. 什么是删除无用赋值？
21. 什么是代码外提？
22. 什么是强度削弱？
23. 什么是合并常量？
24. 什么是删除归纳变量？
25. 什么是基本块的 DAG？
26. 从 DAG 角度看，四元式有哪些类型？画出其对应的 DAG。
27. 简述 DAG 优化的基本思想。
28. 试画出 DAG 优化算法的框架图。
29. 对如下代码进行 DAG 优化：

$100.(+, x, y, \$1)$	$103.(-, x, y, \$1)$	$106.(+, \$1, \$2, u)$
$101.(-, x, y, \$2)$	$104.(+, \$1, \$3, \$2)$	$107.(+, \$1, \$2, v)$
$102.(*, \$1, \$2, \$3)$	$105.(+, \$1, \$2, \$3)$	

30. 对如下代码进行 DAG 优化：

$100.(+, x, y, \$1)$	$103.(+, x, y, \$3)$	$106.(+, \$1, u, v)$
$101.(-, x, y, \$2)$	$104.(-, x, y, \$4)$	$107.(+, \$2, v, u)$
$102.(*, \$1, \$2, u)$	$105.(*, \$3, \$4, v)$	$108.(*, \$3, \$4, u)$

31. 对如下代码进行 DAG 优化：

$100.(+, 1, x, \$1)$	$103.(=, 1, -, x)$	$106.(+, \$3, 1, u)$
$101.(*, 2, x, \$2)$	$104.(+, x, y, \$3)$	$107.(+, x, 1, v)$
$102.(*, \$1, \$2, \$3)$	$105.(+, \$3, x, \$4)$	$108.(j <, x, v, 111)$

32. 对如下代码进行 DAG 优化：

$100.(+, 1, x, \$1)$	$103.(=, 1, -, x)$	$106.(+, \$3, \$4, u)$
$101.(*, 2, x, \$2)$	$104.(=, 2, -, y)$	$107.(+, x, 1, v)$
$102.(*, \$1, \$2, \$3)$	$105.(+, x, y, \$4)$	$108.(j <, u, v, 111)$

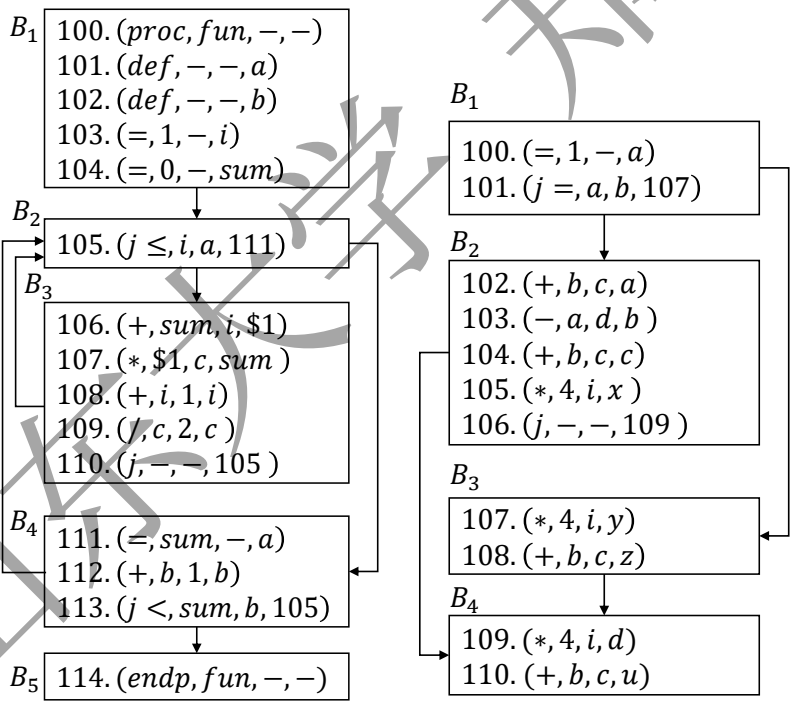
33. 对如下代码进行 DAG 优化：

$100.(\&, a, -, \$1)$	$103.(= [], \$1, \$2, u)$	$106.(= [], \$1, \$2, v)$
$101.(*, 4, i, \$2)$	$104.([] =, 0, x, \$1)$	$107.([] =, \$3, y, \$1)$
$102.(= [], \$1, \$2, \$3)$	$105.(= [], \$1, \$2, \$4)$	$108.(= [], \$1, \$2, w)$

34. 试写出任意路径前向流分析的数据流方程。
35. 试写出任意路径反向流分析的数据流方程。
36. 试写出全路径前向流分析的数据流方程。
37. 试写出全路径反向流分析的数据流方程。
38. 什么是数据流方程解的悲观观点和乐观观点？
39. 试说明什么操作会取消变量的初始化。

山东大学 郑艳伟

40. 什么是可用表达式？什么是非常忙表达式？
41. 变量未初始化检测和变量初始化检测，分别属于哪里数据流分析？
42. 什么是到达定制和 UD 链？
43. UD 链的用途有哪些？
44. 什么是 UD 链和 DU 链？
45. 什么是死变量？如何删除死变量？
46. 什么是循环不变运算？
47. 什么是循环的前置结点？
48. 循环外提的条件有哪些？
49. 什么是归纳变量和基本归纳变量？
50. 图8.3(a)中包含未初始化变量：
    (1) 进行活跃变量分析，计算每个基本块入口和出口处的活跃变量集；
    (2) 进行未初始化变量检测，计算每个基本块入口和出口处的未初始化变量集。
51. 图8.3(b)中包含多个公共子表达式：
    (1) 进行可用表达式分析，计算每个基本块入口和出口处的可用表达式集；
    (2) 进行非常忙表达式分析，计算每个基本块入口和出口处的非常忙表达式集。



(a) 含有未初始化变量流图　　(b) 含有公共子表达式流图

图 8.3 数据流分析流图

52. 图8.4是求水仙花数的流图：
    (1) 进行活跃变量分析，计算每个基本块入口和出口处的活跃变量集；
    (2) 进行未初始化变量检测，计算每个基本块入口和出口处的未初始化变量集；
    (3) 进行可用表达式分析，计算每个基本块入口和出口处的可用表达式集；
    (4) 进行非常忙表达式分析，计算每个基本块入口和出口处的非常忙表达式集。

山东大学 郑艳伟

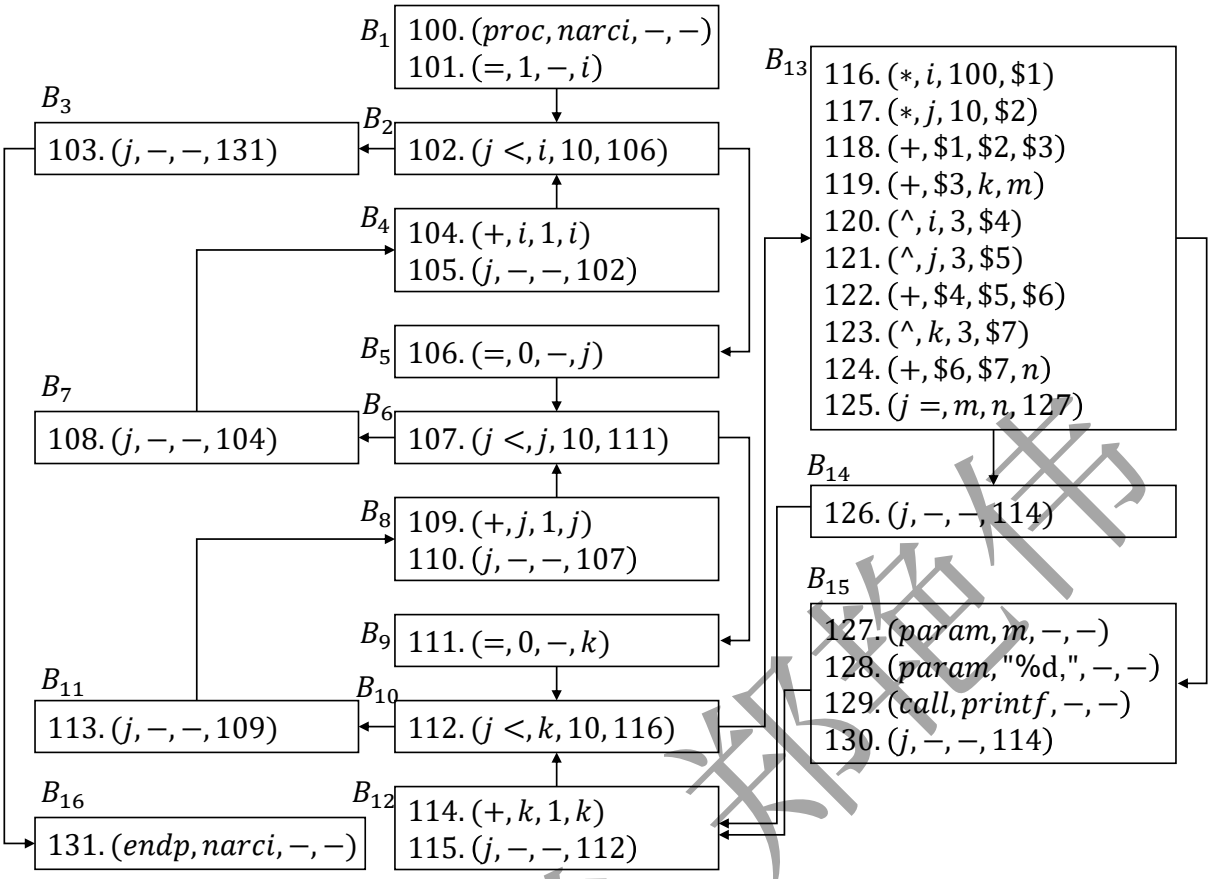


图 8.4 水仙花数流图

53. 对流图8.3(a)：
    (1) 进行到达定值分析；
    (2) 计算 UD 链；
    (3) 进行带引用点的活跃变量分析；
    (4) 计算 DU 链。
54. 对流图8.3(b)：
    (1) 进行到达定值分析；
    (2) 计算 UD 链；
    (3) 进行带引用点的活跃变量分析；
    (4) 计算 DU 链。
55. 对流图8.4：
    (1) 进行到达定值分析；
    (2) 计算 UD 链；
    (3) 进行带引用点的活跃变量分析；
    (4) 计算 DU 链。
56. 对流图8.5(a)：
    (1) 进行到达定值分析；
    (2) 计算 UD 链；
    (3) 进行带引用点的活跃变量分析；

山东大学 郑德伟

(4) 计算 DU 链。

57. 对流图8.5(b)：

(1) 进行到达定值分析；

(2) 计算 UD 链；

(3) 进行带引用点的活跃变量分析；

(4) 计算 DU 链。

58. 对流图8.5(a)：

(1) 进行复写传播和常量传播。

(2) 识别归纳变量；

(3) 删除归纳变量。

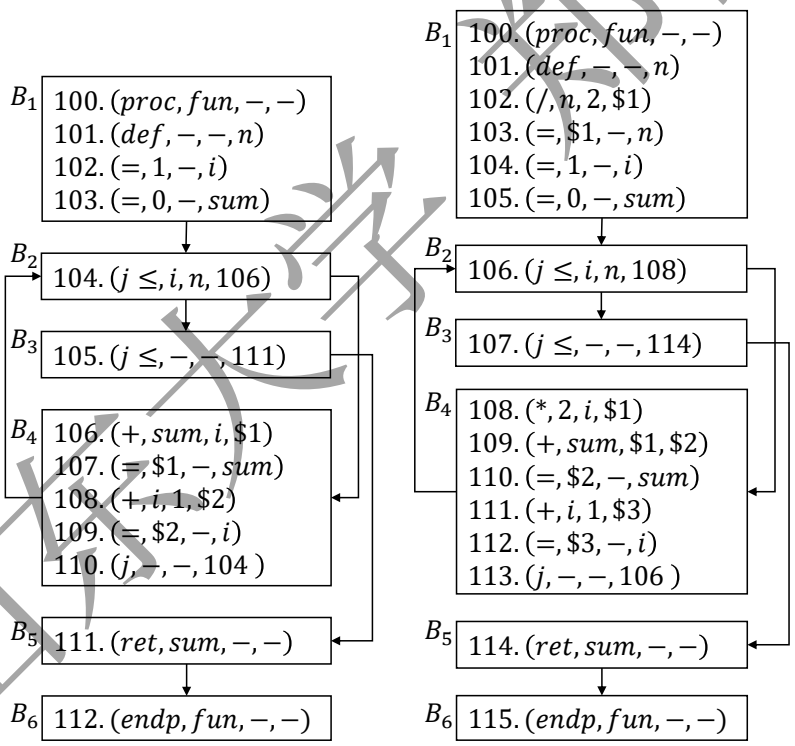
59. 对流图8.5(b)：

(1) 进行复写传播和常量传播。

(2) 识别归纳变量；

(3) 进行强度削弱优化；

(4) 删除归纳变量。



(a) 求和　　(b) 偶数和

图 8.5 单循环

60. 对流图8.6：

(1) 进行代码提升；

(2) 删除公共子表达式；

(3) 删除无用赋值；

(4) 进行复写传播和常量传播。

山东大学 郑艳伟

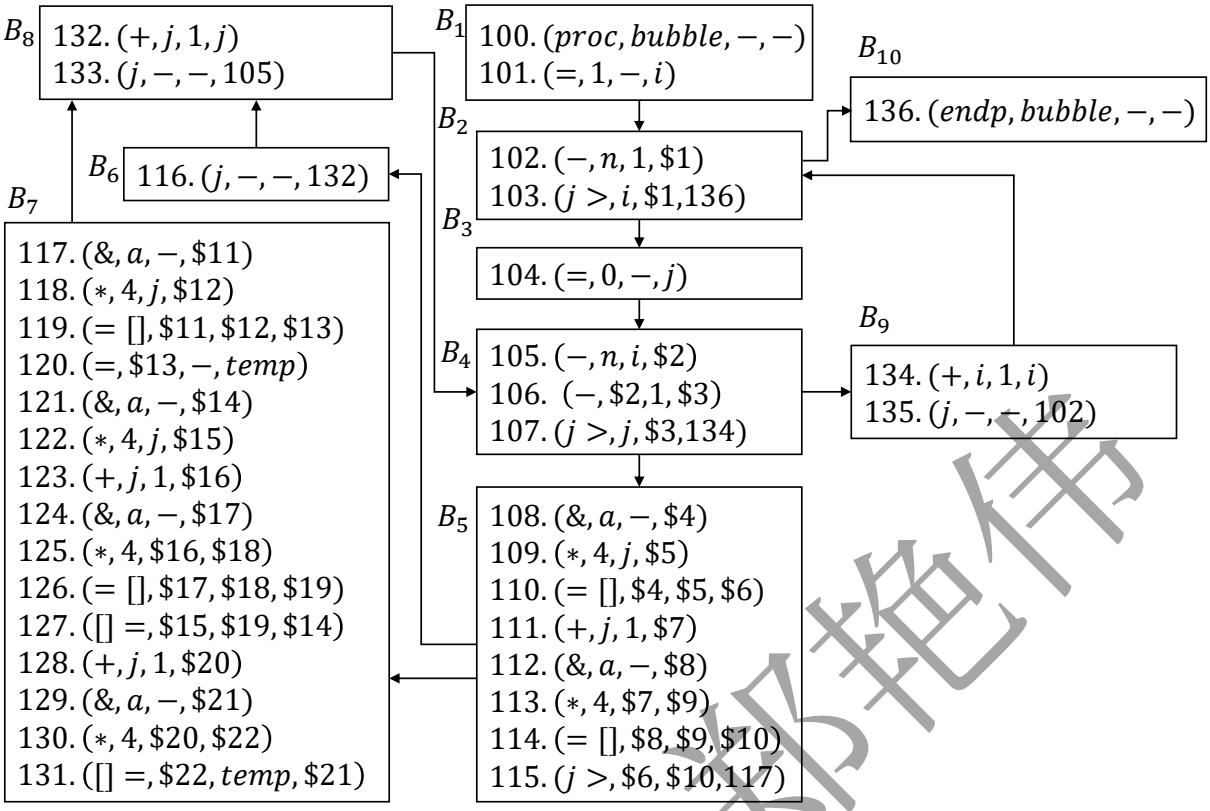


图 8.6 冒泡排序

61. 对流图8.3(a)，检测未初始化变量。
62. 对流图8.3(b)，删除公共子表达式。
63. 对流图8.6：
    (1) 进行代码外提；
    (2) 识别归纳变量；
    (3) 进行强度削弱优化；
    (4) 删除归纳变量。

山东大学 郑艳伟

第 9 章 目标代码生成

第 9 章 练习

1. 概述目标代码生成面临的三大挑战。
2. 指令选择受哪些因素影响?
3. 通过指令调度提升代码效率的考虑有哪些?
4. 简述寄存器分配的发展过程。
5. 什么是待用信息？如何表示？
6. 有如下中间代码，假设可用寄存器为 R_0 和 R_1，u 是基本块出口处活跃变量。

100.$(-, x, y, \$1)$	102.$(+, b, 1, \$3)$	104.$(*, \$3, \$4, \$5)$
101.$(*, a, \$1, \$2)$	103.$(-, x, y, \$4)$	105.$(/, \$2, \$5, u)$

   (1) 求出待用信息和活跃信息；

   (2) 用简单代码生成算法生成目标代码，并写出代码生成过程中的寄存器描述符和地址描述符。

7. 有如下中间代码，假设可用寄存器为 R_0 和 R_1，u 是基本块出口处活跃变量。

100.$(+, x, y, \$1)$	102.$(*, \$1, \$2, \$3)$	104.$(*, \$4, 2, \$5)$
101.$(-, x, y, \$2)$	103.$(=, \$3, -, \$4)$	105.$(=, \$5, -, u)$

   (1) 求出待用信息和活跃信息；

   (2) 用简单代码生成算法生成目标代码，并写出代码生成过程中的寄存器描述符和地址描述符。

8. 写出加、减、乘、除、负等整型运算、以及赋值运算四元式对应的目标代码。
9. 写出浮点运算四元式对应的目标代码。
10. 写出数组运算四元式对应的目标代码。
11. 写出转移语句四元式对应的目标代码。
12. 写出地址和指针四元式对应的目标代码。
13. 写出类型转换四元式对应的目标代码。
14. 写出程序和过程四元式对应的目标代码。
15. 写出如下指令的执行代价。

100.mov ebp, esp	102.mov [ebp + 8], eax	104.imul eax, ebx
101.mov eax, [ebp + 8]	103.add eax, ebx	105.sub eax, [ebp - 20]

16. 写出如下指令的执行代价。

100.lea eax, [ebp - 20]	103.cmp eax, [ebp - 20]	106.jg L
101.inc eax	104.cmp [ebp - 20], ebx	107.push eax
102.inc [ebp - 20]	105.jmp L	108.pop ebp

17. 写出如下指令的执行代价。

100.fld [ebp - 20]	103.fcomp [ebp - 20]	106.faddp st(0)
101.fld1	104.fnstsw ax	107.faddp [ebp - 20]
102.fabs	105.sahf	

山东大学 郑楚伟

18. 有如下中间代码，假设可用寄存器为 R_0 和 R_1，u 是基本块出口处活跃变量。

$100.(-, x, y, \$1)$	$102.(+, \$2, \$1, \$3)$	$104.(*, \$3, 2, \$5)$
$101.(*, \$1, z, \$2)$	$103.(+, \$1, \$3, \$4)$	$105.(*, \$4, \$5, u)$

(1) 用简单代码生成算法生成目标代码；
(2) 使用 DAG 目标代码优化算法对四元式重新排序；
(3) 用简单代码生成算法生成重新排序后的目标代码；
(4) 比较优化前后目标代码的优劣。

19. 有如下中间代码，假设可用寄存器为 R_0 和 R_1，u 是基本块出口处活跃变量。

$100.(+, x, y, \$1)$	$102.(*, \$1, 2, \$3)$	$104.(*, \$3, \$4, \$5)$
$101.(-, x, y, \$2)$	$103.(*, \$2, 2, \$4)$	$105.(-, \$5, 1, u)$

(1) 用简单代码生成算法生成目标代码；
(2) 使用 DAG 目标代码优化算法对四元式重新排序；
(3) 用简单代码生成算法生成重新排序后的目标代码；
(4) 比较优化前后目标代码的优劣。

20. 有如下中间代码，假设可用寄存器为 R_0 和 R_1，u 是基本块出口处活跃变量。

$100.(+, x, y, \$1)$	$102.(*, \$1, \$2, \$3)$	$104.(+, \$3, \$4, \$5)$
$101.(-, x, y, \$2)$	$103.(*, \$2, \$2, \$4)$	$105.(-, \$5, 1, u)$

(1) 使用 DAG 对中间代码进行优化；
(2) 使用 DAG 目标代码优化算法对四元式重新排序；
(3) 用简单代码生成算法生成重新排序后的目标代码。

21. 有如下中间代码，假设可用寄存器为 R_0 和 R_1，u、v 是基本块出口处活跃变量。

$100.(+, x, 1, \$1)$	$103.(=, 1, -, x)$	$106.(+, \$3, \$4, u)$
$101.(-, x, 1, \$2)$	$104.(=, 2, -, y)$	$107.(+, x, 1, v)$
$102.(*, \$1, \$2, \$3)$	$105.(+, x, y, \$4)$	$108.(j <, u, v, 111)$

(1) 使用 DAG 对中间代码进行优化；
(2) 使用 DAG 目标代码优化算法对四元式重新排序；
(3) 用简单代码生成算法生成重新排序后的目标代码。

22. 如图9.1(a)所示的流图，两个循环 $B_2 \to B_3 \to B_5 \to B_2$ 和 $B_2 \to B_4 \to B_5 \to B_2$ 有相同的结点。在固定分配寄存器给循环中变量时，将两个循环节省的执行代价相加，作为变量固定分配寄存器后节省的指令代价。
(1) 进行活跃变量分析；
(2) 计算每个变量在固定分配寄存器后节省的指令代价；
(3) 假设将寄存器 EAX、EBX、EDX 固定分配给 3 个循环中的变量，选出这 3 给变量；
(4) 其他寄存器，包括 ECX、ESI 和 EDI，可以分配给其他变量，用简单代码生成算法生成目标代码。

23. 如图9.1(b)所示的流图，循环 $B_2 \to B_3 \to B_2$ 是循环 $B_2 \to B_3 \to B_4 \to B_2$ 和循环 $B_2 \to B_4 \to B_2$ 的内层嵌套循环。由于内层循环次数多，在固定分配寄存器给循环中变量时，使用加权求和计算节省的执行代价。如有 n 个循环（本题有 3 个循环），采用 $cost(x) = \sum_{i=1}^{n} cost_i(x) \times 10^{d_i - 1}$ 计算变量 x 节省的总代价，其中 $cost_i(x)$ 为第 i 个循环对变量 x 节省的代价，d_i 为该循环的深度。

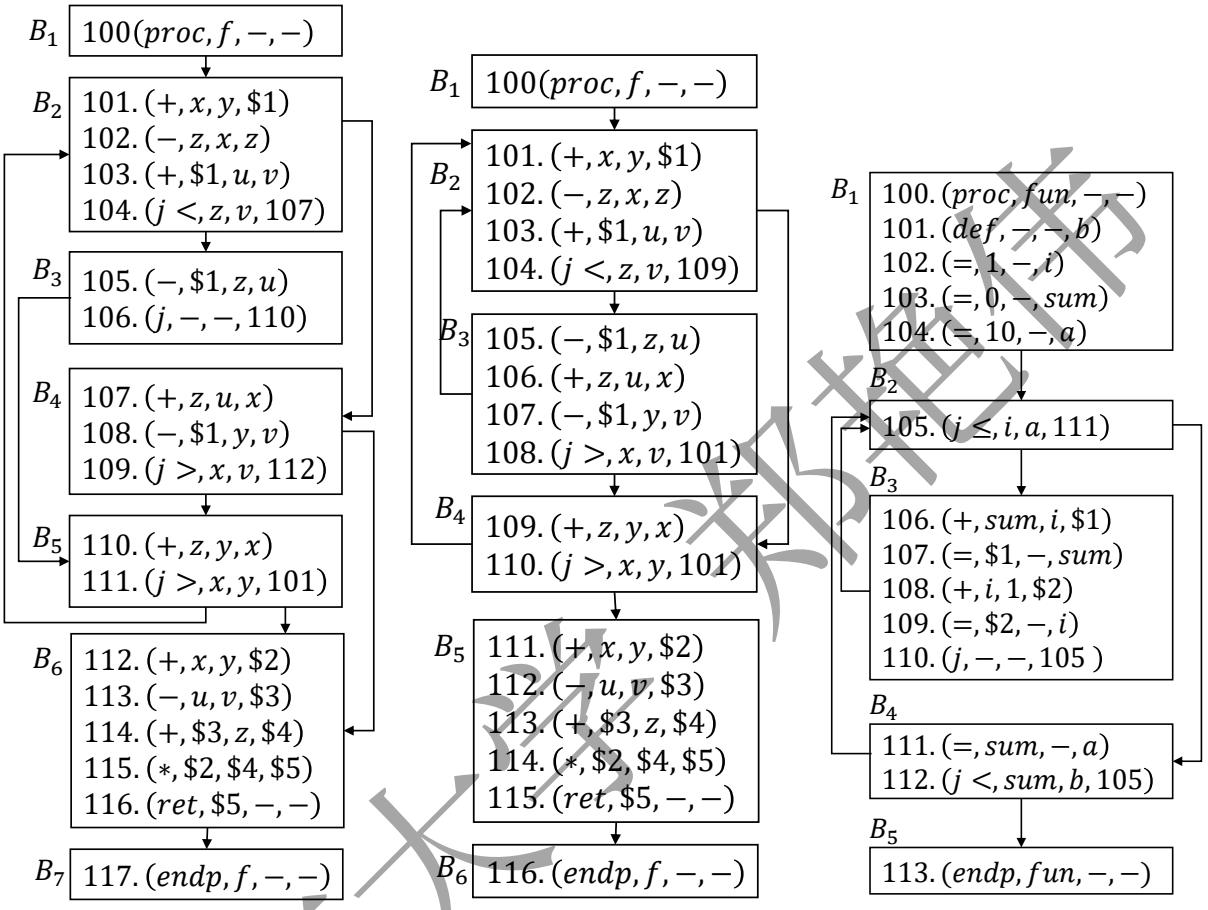
山东大学 郑艳伟

(1) 进行活跃变量分析；

(2) 计算每个变量在固定分配寄存器后节省的指令代价；

(3) 假设将寄存器 EAX、EBX、EDX 固定分配给 3 个循环中的变量，选出这 3 给变量；

(4) 其他寄存器，包括 ECX、ESI 和 EDI，可以分配给其他变量，用简单代码生成算法生成目标代码。



(a) 多循环流图　(b) 嵌套循环流图 1　(c) 嵌套循环流图 2

图 9.1 循环寄存器分配流图

24. 如图9.1(c)所示的流图，循环 $B_2 \to B_3 \to B_2$ 是循环 $B_2 \to B_4 \to B_2$ 的内层嵌套循环。由于内层循环次数多，在固定分配寄存器给循环中变量时，使用加权求和计算节省的执行代价。如有 n 个循环（本题有 2 个循环），采用 $cost(x) = \sum_{i=1}^{n} cost_i(x) \times 10^{d_i - 1}$ 计算变量 x 节省的总代价，其中 $cost_i(x)$ 为第 i 个循环对变量 x 节省的代价，d_i 为该循环的深度。

    (1) 进行活跃变量分析；

    (2) 计算每个变量在固定分配寄存器后节省的指令代价；

    (3) 假设将寄存器 EAX、EBX、EDX 固定分配给 3 个循环中的变量，选出这 3 给变量；

    (4) 其他寄存器，包括 ECX、ESI 和 EDI，可以分配给其他变量，用简单代码生成算法生成目标代码。

25. DAG 中，结点的附加是该结点的定值变量；结点 n_2 是 n_1 的父结点，说明 n_2 的定值语句要引用 n_1 代表的变量。根据这些信息，试设计由 DAG 计算各中间代码待用信息的算法。

26. 调用 C 运行时库打印函数 printf 和 scanf 的第一个参数是常量字符串，用 *cstr* 表示，那么会生

山东大学 郑艳伟

成四元式 $(param, cstr, -, -)$。该指令的常量字符串 $cstr$ 中，用户输入的转义字符如 \r、\n、\t，已由词法分析器转换为真实字符 0Dh、0Ah、09h，但没有表示最后字符串结束的字符 0。

(1) 试设计 $cstr$ 的压栈算法，需要将字节对齐到 4 的倍数；

(2) 设计过程调用完成后，清理 $cstr$ 占用栈帧空间的算法（涉及整个过程调用算法的修改）。

27. 画图表示图着色寄存器分配算法的框架，并简述各部分功用。
28. 分配虚拟寄存器的目的是什么？
29. 简述低级中间代码表示中的数据项及其含义。
30. 什么是寄存器的冲突图？构造的基本方法是怎样的？
31. 简述寄存器冲突图构建中，x86 的除法指令中 EDX:EAX 的寄存器组合如何处理？
32. 什么是合并虚拟寄存器？具体如何操作？
33. 图着色寄存器分配中，邻接表的作用是什么？简述其数据结构。
34. 图着色寄存器分配中的溢出代价是什么？包括哪几个方面？
35. 什么是图着色寄存器分配中的修剪冲突图？简述其规则。
36. 什么是寄存器压力？如何减少寄存器压力？
37. 使用 x86 的 EAX、EBX、ECX、EDX 共 4 个寄存器，对流图9.1(b)：

    (1) 进行图着色寄存器分配；

    (2) 根据寄存器分配结果生成目标代码。
38. 使用 x86 的 EAX、EBX、ECX、EDX 共 4 个寄存器，对流图9.1(c)：

    (1) 进行图着色寄存器分配；

    (2) 根据寄存器分配结果生成目标代码。
39. 使用 x86 的 6 个通用寄存器，对流图9.2：

    (1) 进行图着色寄存器分配；

    (2) 根据寄存器分配结果生成目标代码。

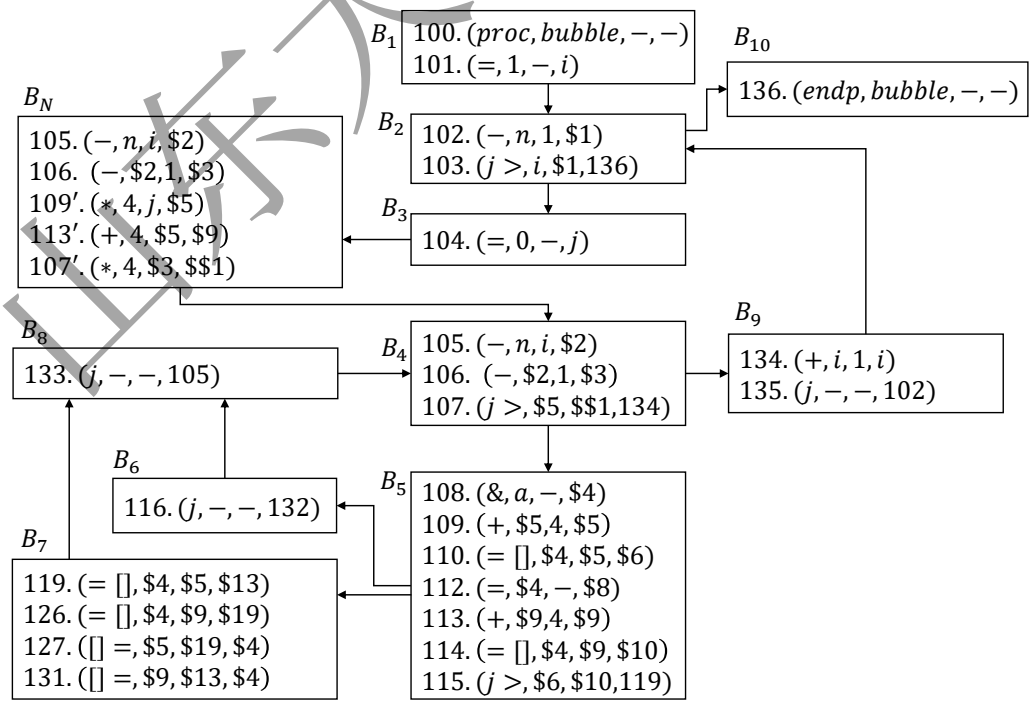


图 9.2 合并常量和删除归纳变量后的流图

山东大学 郑艳伟

40. 试改进图着色的溢出算法，如果一个溢出变量在同一个基本块中使用多次，并且在块中最后一次使用前没有被杀死，则这个值在此基本块中只需要加载一次。
41. 什么是流图线性化？简述其规则。
42. 什么是活跃区间？
43. 使用 x86 的 EAX、EBX、ECX、EDX 共 4 个寄存器，对流图9.1(b)：
    (1) 进行线性扫描寄存器分配；
    (2) 根据寄存器分配结果生成目标代码。
44. 使用 x86 的 EAX、EBX、ECX、EDX 共 4 个寄存器，对流图9.1(c)：
    (1) 进行线性扫描寄存器分配；
    (2) 根据寄存器分配结果生成目标代码。
45. 使用 x86 的 6 个通用寄存器，对流图9.2：
    (1) 进行线性扫描寄存器分配；
    (2) 根据寄存器分配结果生成目标代码。
46. 试构造类型转换指令的全局目标代码生成算法，需要支持操作数为常数的情形。
47. 写出如下代码窥孔优化后的代码。

100. add eax, 0	103. add eax, ebx	106. pop ebp
101. cmp -1, 0	104. L1:	107. L2: ...
102. jl L2	105. imul eax, 1	

48. 写出如下代码窥孔优化后的代码。

100. cmp eax, ebx	103. L1:	106. pop ebp
101. jl L1	104. add ecx, edx	107. L2: ...
102. jmp L2	105. imul eax, 1	

49. 有时会遇到一个值被保存到主存单元后，又立即被取到一个寄存器的情况。即指令 “mov a, R_0” 后紧跟着一个指令 “mov R_1, a”，其中寄存器 R_0 和 R_1 可能是，也可能不是同一个寄存器。请设计通过窥孔优化，对这一对指令进行优化的算法。
50. 移位指令计算比乘法指令快的多。如果乘数是一个常数，如 “imul $R_0, 70$”，由于 $70 = 2^6 + 2^2 + 2^1$，可以将乘法指令修改为如下 6 条指令：

1. mov R_1, R_0	3. shl $R_1, 2$	5. shl $R_1, 4$
2. shl $R_0, 1$	4. add R_0, R_1	6. add R_0, R_1

基本思路是借用另外一个寄存器 R_1，最低位的左移 1 位在 R_0 中完成，剩余左移 2、6 位（左移 2 位基础上再左移 4 位）在 R_1 中完成，并累加到 R_0。在一般的系统中，如果一个常数能分解到不超过 3 项 2 的幂相加，这个优化就是有效的。这种优化的限制是将无符号乘法使用符号乘法指令计算，乘法不会溢出使最高位为 1。试设计这种优化算法。（提示：把 70 写成二进制数，其第 1、2、6 位为 1，其余位为 0；或者连续除以 2 求余，次数从 0 算起，第 1、2、6 次除以 2 的余数为 1。）

山东大学 郑艳伟

第 10 章 面向对象语言的翻译

第 10 章 练习

1. 简述面向对象中属性访问的规则。
2. 简述面向对象中方法调用的规则。
3. 什么是类层次表？它与 Pascal 中嵌套层次表的联系与区别是什么？
4. 对代码10.1，画出运行时类层次表、各属性在存储空间的安排示意图。

代码 10.1 类继承

```
public A : Object {
  int x, y;
  A(int x, int y) { this.x = x; this.y = y; }
  void f1(int a, int b){...}
  double f2(double a, double b) {...}
}
public B : A {
  double sum;
  B(double sum) : A(0, 100) { this.sum = sum; }
  double f2(double a, double b) {...}
}
```

5. 简述面向对象中符号表如何组织。
6. 简述符号表查找各种不同情形的规则。
7. 画出代码10.1的类符号表。
8. 画出代码10.1的类方法表。
9. 请设计一种合理的静态属性的运行时存储和访问策略。
10. 简述对象赋值的各种情形和规则。
11. 简述类型检查的各种情形，并画图表示其执行逻辑。
12. 什么是相关类方法集合？如何获得？
13. 简述单继承中属性引用 $id1.id2$ 的执行逻辑。
14. 简述单继承中方法调用 $id1.id2()$ 的执行逻辑。
15. 简述单继承中方法调用 $id1 :: id2()$ 的执行逻辑。
16. 试翻译代码10.1的声明语句，填写符号表。
17. 根据代码10.1，有声明语句：A a1, a2; B b; 翻译语句：a1 = new A(30, 40); a2 = a1; free a2;
18. 根据代码10.1，有声明语句：A a; B b; 翻译语句：if (a.typeof(B)) b = a;
19. 根据代码10.1，有声明语句：A a; B b; 翻译语句：if (typeid(a) = typeid(B)) b = a;
20. 根据代码10.1，有声明语句：A a; B b, double t; 翻译语句：a.f1(3, 4); t = b.f2(1.0, 2.0);

山东大学 郑艳伟

参考文献

[1] ALFRED V A, MONICA S L, RAVI S, et al. Compilers: Principles, Techniques, & Tools[M]. 2nd ed. Noida: Pearson Education India, 2007.

[2] ANDREW W A, MAIA G. Modern Compiler Implementation in C[M]. Cambridge: Cambridge university press, 2004.

[3] ASHOURI A H, KILLIAN W, CAVAZOS J, et al. A Survey on Compiler Autotuning Using Machine Learning[J]. ACM Computing Surveys (CSUR), 2018, 51(5): 1-42.

[4] BACON D F, GRAHAM S L, SHARP O J. Compiler Transformations for High-Performance Computing[J]. ACM Computing Surveys (CSUR), 1994, 26(4): 345-420.

[5] BERNSTEIN D, GOLUMBIC M, MANSOUR Y, et al. Spill Code Minimization Techniques for Optimizing Compliers[J]. ACM SIGPLAN Notices, 1989, 24(7): 258-263.

[6] BOOCH G. The Unified Modeling Language User Guide[M]. Pearson Education India, 2005.

[7] BRIGGS P. Register Allocation via Graph Coloring[M]. Rice University, 1992.

[8] LOPES B C, AULER R. Getting Started with LLVM Core Libraries[M]. Packt Publishing Ltd, 2014.

[9] CAMPBELL E. Random Compiler for Fast Hamiltonian Simulation[J]. Physical review letters, 2019, 123(7): 070503.

[10] CHAITIN G J, AUSLANDER M A, CHANDRA A K, et al. Register Allocation via Coloring[J]. Computer languages, 1981, 6(1): 47-57.

[11] 陈火旺, 刘春林, 谭庆平, 等. 程序设计语言编译原理[M]. 3 版. 北京: 国防工业出版社, 2000.

[12] 陈意云. 编译原理和技术[M]. 2 版. 合肥: 中国科学技术大学出版社, 2002.

[13] 陈有祺. 形式语言与自动机[M]. 北京: 机械工业出版社, 2008.

[14] 陈意云, 张昱. 编译原理[M]. 3 版. 北京: 高等教育出版社, 2014.

[15] CHEN J, PATRA J, PRADEL M, et al. A Survey of Compiler Testing[J]. ACM Computing Surveys (CSUR), 2020, 53(1): 1-36.

[16] CHRISTENSEN H K, BRODAL G S. Algorithms for Finding Dominators in Directed Graphs[D]. Aarhus Universitet, Datalogisk Institut, 2016.

[17] COELHO H, COTTA J. Prolog by Example: How to Learn, Teach, and Use It[M]. New York: Springer-Verlag, 1988.

[18] COOPER K D, TORCZON L. Engineering a Compiler[M]. Elsevier, 2011.

[19] 段刚. 加密与解密[M]. 4 版. 北京: 电子工业出版社, 2018.

[20] FERRARI D, CACCIAPUOTI A S, AMORETTI M, et al. Compiler Design for Distributed Quantum Computing[J]. ArXiv preprint arXiv:2012.09680, 2020.

山东大学 郑艳伟

[21] DAVID W F C R H. A Retargetable C Compiler: Design and Implementation[M]. New York: Pearson Education, Inc., 1995.

[22] GRUNE D, VAN REEUWIJK K, BAL H E, et al. Modern Compiler Design[M]. Berlin: Springer Science & Business Media, 2012.

[23] 韩太鲁, 姚栋义, 董吉文, 等. 编译原理[M]. 东营: 石油大学出版社, 2002.

[24] 黄贤英, 王柯柯, 曹琼, 等. 编译原理及实践教程[M]. 3 版. 北京: 清华大学出版社, 2019.

[25] IRVINE K R, DAS L B. Assembly Language for X86 Processors[M]. 7th ed. New York: Pearson Education, Inc., 2015.

[26] 蒋立源, 康慕宁. 编译原理[M]. 2 版. 西安: 西北工业大学出版社, 2000.

[27] JOHNSON S C, et al. Yacc: Yet Another Compiler-Compiler[M]. New York: Bell Laboratories Murray Hill, NJ, 1975.

[28] KING K N. C Programming: A Modern Approach[M]. 2nd ed. New York: WW Norton & company, 2021.

[29] KOEPLINGER D, FELDMAN M, PRABHAKAR R, et al. Spatial: A Language and Compiler for Application Accelerators[C]//Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation. 2018: 296-311.

[30] LI M, LIU Y, LIU X, et al. The Deep Learning Compiler: A Comprehensive Survey[J]. IEEE Transactions on Parallel and Distributed Systems, 2020, 32(3): 708-727.

[31] 李兴华. Java 编程从入门到实践[M]. 北京: 中国水利水电出版社, 2021.

[32] LIPPMAN S B. Essential C++[M]. New York: Pearson Education, Inc., 2002.

[33] 马希文, 宋柔. LISP 语言[M]. 北京: 高等教育出版社, 1990.

[34] MATTHES E. Python Crash Course: A Hands-on, Project-Based Introduction to Programming[M]. no starch press, 2023.

[35] MICHAELIS M. Essential C# 8.0[M]. 7th ed. Boston: Addison-Wesley, 2020.

[36] MUCHNICK S, et al. Advanced Compiler Design and Implementation[M]. San Francisco: Morgan kaufmann, 1997.

[37] POLETTO M, SARKAR V. Linear Scan Register Allocation[J]. ACM Transactions on Programming Languages and Systems (TOPLAS), 1999, 21(5): 895-913.

[38] 钱林松, 张延清. C++ 反汇编与逆向分析技术揭秘[M]. 2 版. 北京: 机械工业出版社, 2021.

[39] 青木峰郎. 自制编译器[M]. 北京: 人民邮电出版社, 2016.

[40] RASTELLO F, TICHADOU F B. SSA-Based Compiler Design[M]. Springer Nature, 2022.

[41] REIZER N R, ABOWD G D, MEYERS B C, et al. Using Formal Methods for Requirements Specification of a Proposed POSIX Standard[C]//Proceedings of IEEE International Conference on Requirements Engineering. 1994: 118-125.

[42] C.MARTIN R, 韩磊（译）. 代码整洁之道[M]. 人民邮电出版社, 2020.

山东大学 郑艳伟

[43] SILBERSCHATZ A, GALVIN P B, GAGNE G. Operating System Concepts Essentials[M]. 9th ed. New Jersey: John Wiley & Sons, Inc., 2013.

[44] LIPPMAN S B, LAJOIE J, MOO B E. C++ Primer[M]. 5th ed. 北京: 电子工业出版社, 2013.

[45] STASHCHUK B. JavaScript Bible-JavaScript and ES6 Bootcamp 2019[M]. Packt Publishing, 2019.

[46] 谭浩强. C 语言程序设计[M]. 4 版. 北京: 清华大学出版社, 2020.

[47] TROFIN M, QIAN Y, BREVDO E, et al. Mlgo: A Machine Learning Guided Compiler Optimizations Framework[J]. ArXiv preprint arXiv:2101.04808, 2021.

[48] WAITE W M, GOOS G. Compiler Construction[M]. Berlin: Springer Science & Business Media, 2012.

[49] 王珊, 萨师煊. 数据库系统概论[M]. 4 版. 北京: 高等教育出版社, 2006.

[50] 王晓斌, 陈文宇, 余盛季, 等. 程序设计语言与编译–语言的设计和实现[M]. 4 版. 北京: 电子工业出版社, 2015.

[51] 王爽. 汇编语言[M]. 4 版. 北京: 清华大学出版社, 2019.

[52] WATT D. PASCAL: Pocket Guide[M]. London: Pitman, 1982.

[53] 文全刚, 张荣高. 汇编语言程序设计——基于 ARM 体系结构[M]. 4 版. 北京: 北京航空航天大学出版社, 2021.

[54] WICHMANN B A. ALGOL 60 Compilation and Assessment[M]. New York: Academic Press, 1973.

[55] KENNEDY W, KETELSEN B, MARTIN E S. Go in Action[M]. Greenwich: Manning Publications, 2015.

[56] WIMMER C. Linear Scan Register Allocation for the Java HotSpot Client Compiler. Master's Thesis[J]. Institute for System Software, Johannes Kepler University Linz, 2004.

[57] 严蔚敏, 吴伟民. 数据结构（C 语言版）[M]. 北京: 清华大学出版社, 2019.

[58] 张素琴, 吕映芝, 蒋维杜, 等. 编译原理[M]. 2 版. 北京: 清华大学出版社, 2005.

[59] 张昱, 陈意云. 编译原理实验教程[M]. 北京: 高等教育出版社, 2009.

[60] 张莉, 杨海燕, 史晓华, 等. 编译原理及编译程序构造[M]. 北京: 清华大学出版社, 2011.

[61] 左孝凌, 李为鉴, 刘永才. 离散数学[M]. 上海: 上海科学技术文献出版社, 1982.

山东大学 郑艳伟